

Herramientas de Programación Web del Servidor: Desarrollo del Backend con PHP 8.x y ASP.NET Core / Java Spring Boot

Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación

Integrantes:

FIGUEROA MORALES BRYAN JAVIER

ROMERO MENDEZ BRYAM STEVEN

VELEZ LOPEZ RICARDO ELIAS

Asignatura: Aplicaciones Web

Carrera: Software

Nivel / Paralelo: 5to Nivel – A

Docente responsable: Guerrero Ulloa Gleiston Ciceron

Periodo académico 2026-2027 PPA

Quevedo – Los Ríos – Ecuador

Repositorio GitHub: [Click aqui](#)

1. Introducción

El presente informe documenta el progreso de la práctica experimental de la Unidad II para la materia Aplicaciones Web. La finalidad es edificar el *backend* del Proyecto Fin de Curso (PFC) utilizando, como mínimo, dos tecnologías de programación en el lado del servidor: PHP 8.x, de carácter obligatorio, y una segunda tecnología seleccionada entre ASP.NET Core o Java con Spring Boot. La labor incluye la aplicación de un sistema de autenticación seguro, un módulo CRUD que se fundamenta en el patrón Repository y tiene acceso a datos a través de PDO y *prepared statements*, así como la implementación de acciones para mitigar las fallas más significativas del OWASP Top 10.

La razón detrás de esta práctica es una necesidad específica de formación: en el trabajo profesional del desarrollo web, generalmente no es suficiente con conocer un solo lenguaje o marco. Las organizaciones analizan a los equipos de desarrollo en función de su habilidad para tomar decisiones informadas sobre arquitectura y comparar tecnologías mediante criterios objetivos, como la seguridad por defecto, el rendimiento, el costo de mantenimiento y la disponibilidad del talento. Por ello, el informe no se restringe a una puesta en marcha funcional, sino que incluye un análisis comparativo exhaustivo entre PHP y la segunda tecnología elegida.

2. Fundamento Teórico

2.1. Ciclo de vida del servidor web y arquitecturas de integración

El rendimiento, la escalabilidad y el uso de memoria de una aplicación dinámica dependen directamente del modo en que un lenguaje de programación se acopla con el servidor web. El Common Gateway Interface (CGI) es el modelo más antiguo. En él, la solicitud HTTP genera un nuevo proceso del sistema operativo en su totalidad: el servidor transmite los datos de la petición a través de las variables de entorno y el cuerpo de la solicitud por medio de la entrada estándar (stdin); por su parte, el *script* coloca la respuesta —las cabeceras HTTP incluidas— en la salida estándar (stdout) [1].

El precio de este modelo es alto: generar y eliminar un proceso por cada petición supone una sobrecarga en la inicialización del intérprete, lo que se vuelve inasequible con carga concurrente. Por eso fue reemplazado en la práctica por arquitecturas persistentes [2,3].

Una primera transformación fueron los módulos embebidos en el servidor mismo, como `mod_php` para Apache. En estos casos, el intérprete de PHP se carga solamente una vez dentro del proceso del servidor y persiste en la memoria entre las peticiones, compartiendo además un *pool* de hilos o procesos gestionado por el propio servidor web. Esta continuidad disminuye considerablemente la latencia de arranque; sin embargo, sincroniza el ciclo vital del lenguaje con el del servidor: para reiniciar la configuración de PHP

es necesario reiniciar Apache [2,3].

FastCGI, y en particular su versión más actual, PHP-FPM (FastCGI Process Manager), soluciona este acoplamiento al disociar el servidor web (Nginx, por ejemplo) del grupo de procesos que ejecutan PHP. PHP-FPM conserva un grupo de procesos trabajadores que pueden ser configurados, con políticas de escalado dinámico, estático o bajo demanda. Estos procesos reciben las solicitudes por medio del protocolo FastCGI, usando un *socket* Unix o uno de red. Esta arquitectura posibilita la calibración independiente de la cantidad de procesos en función de la carga, y se aconseja utilizarla para PHP 8.x en producción porque ofrece una mejor relación entre el rendimiento (más solicitudes por segundo) y el aislamiento de errores en comparación con `mod_php` [2,3].

Las plataformas Java y .NET han tomado una perspectiva diferente: el servidor HTTP se integra directamente en el proceso de la aplicación. Spring Boot incorpora de forma predeterminada un contenedor de *servlets*, como Tomcat (o también Undertow o Jetty), mientras que ASP.NET Core emplea Kestrel, un servidor asíncrono y multiplataforma optimizado que se fundamenta en la biblioteca de E/S de .NET. En las dos situaciones, la aplicación es un procedimiento autocontenido que escucha directamente en un puerto TCP. Esto facilita el despliegue en contenedores Docker y posibilita un control programático total del *pipeline* de solicitudes, aunque conlleva que el programador asuma responsabilidades (como la terminación TLS o el balanceo) que antes pertenecían al servidor web frontal [1,4].

Desde el punto de vista del rendimiento, la literatura está de acuerdo en que los modelos fundamentados en procesos persistentes (PHP-FPM, Kestrel y Tomcat embebido) sobrepasan en gran medida a CGI clásico en términos de solicitudes por segundo; esto se debe a que eliminan el costo de iniciar el intérprete por cada petición. La diferencia entre PHP-FPM y los servidores embebidos de .NET/Java depende mayoritariamente del tipo síncrono o asíncrono del modelo de concurrencia correspondiente a cada plataforma. En situaciones con alta concurrencia, Kestrel es especialmente eficaz gracias a su modelo no bloqueante de E/S [1,4].

PERL/CGI: contexto histórico y limitaciones actuales En los años noventa, Perl y CGI fueron una de las primeras combinaciones empleadas para crear contenido dinámico en la web. El módulo CGI.pm de Perl, que encapsula el protocolo CGI comentado en la sección previa, ofreció a lo largo de años una abstracción estándar para manejar *cookies*, leer parámetros de formularios y crear cabeceras HTTP. No obstante, a partir de la versión 5.22 se eliminó el mantenimiento de CGI.pm del núcleo de Perl, admitiendo que el ecosistema se había mudado hacia *frameworks* persistentes como `mod_perl`, Plack/PSGI o modelos FastCGI y de servidor embebido en otros lenguajes, los cuales se mencionaron anteriormente.

El deterioro de Perl/CGI como pila para el desarrollo web moderno se debe a factores tanto de

rendimiento (el costo por solicitud para iniciar el intérprete de Perl es similar al del CGI clásico en cualquier lenguaje) como de ecosistema: la falta de un marco dominante similar a ASP.NET Core, Symfony, Spring Boot o Laravel, junto con una sintaxis considerada como menos legible para equipos grandes, ha llevado a que Perl sea relegado hacia nichos especializados. Hoy en día, Perl sigue siendo importante para la gestión de sistemas Unix/Linux (debido a su integración nativa con utilidades de texto y a su disponibilidad global), para la bioinformática (ya que bibliotecas como BioPerl se mantienen activas en los *pipelines* heredados de análisis de secuencias genómicas) y para las funciones de *scripting* y procesamiento de grandes cantidades de texto, ya que cuenta con un motor maduro para expresiones regulares. Hoy en día, su aplicación para crear nuevas aplicaciones web dirigidas al usuario final es insignificante comparado con PHP, JavaScript/Node.js, Python o las plataformas analizadas en esta práctica.

PHP 8.x: características clave para el desarrollo web moderno La serie PHP 8.0-8.3 trajo avances significativos en la expresividad y el rendimiento del lenguaje. El compilador Just-In-Time (JIT), que fue incluido en PHP 8.0, es el más importante en términos de rendimiento; convierte fragmentos del código a instrucciones nativas de máquina durante la ejecución. Aunque el JIT favorece mayormente a tareas de cómputo intensivo (como la compresión, el procesamiento numérico y el *machine learning* embebido), su efecto en aplicaciones web convencionales que están regidas por E/S —por ejemplo, la autenticación y el CRUD de este PFC— no es muy grande, debido a que generalmente la congestión se produce en la base de datos y en la red, no al ejecutar *bytecode* [5].

Con respecto a la expresividad, PHP 8.0 incorporó las `match` expressions, una opción más segura que `switch` porque emplea comparación estricta y no necesita `break`, lo que disminuye errores de caída entre casos; además, incluyó *named arguments* para mejorar la legibilidad de llamadas con varios parámetros opcionales y *union types* para declarar retornos y parámetros con múltiples tipos posibles. En PHP 8.1 se incluyeron los `enums` nativos y las *fibers*, una primitiva de concurrencia cooperativa de nivel bajo que sirve como base para desarrollar bibliotecas de programación asíncrona. Las clases de solo lectura (`readonly classes`) fueron incorporadas en PHP 8.2 y posibilitan declarar que todas las propiedades de una clase son inalterables después de construidas, lo cual respalda los principios del diseño inmutable que se emplean en el patrón Repository mencionado en este informe [5].

Los *benchmarks* divulgados por el proyecto PHP y la comunidad coinciden en que las optimizaciones del motor Zend, así como el JIT en menor medida, hacen que PHP 8.x tenga un mejor rendimiento de entre 2 y 3 veces más que PHP 7.4 para cargas significativas de *frameworks* como WordPress o Laravel. Esta mejora es importante para explicar técnicamente, en la tabla comparativa de este informe, las razones por las que PHP 8.2 continúa siendo una alternativa competitiva frente a plataformas compiladas como Java o .NET [2, 5].

Criterios de selección de tecnología servidor La decisión de elegir entre Java/Spring Boot, PHP y ASP.NET Core para un proyecto real no puede depender solamente de los gustos personales del equipo de desarrollo; es necesario una matriz de decisión que valore criterios técnicos y el contexto del mercado. Entre los criterios más relevantes para el PFC de esta asignatura, considerando además el entorno ecuatoriano, se encuentran: curva de aprendizaje del equipo, disponibilidad y costo del *hosting* en Ecuador, tamaño y actividad de la comunidad, madurez del ecosistema de paquetes (Composer/-Packagist para PHP, NuGet para .NET, Maven Central para Java), rendimiento medido en *benchmarks* independientes, salario promedio de desarrolladores en el mercado local, facilidad de integración con proveedores *cloud* (AWS, Azure, GCP), modelo de licenciamiento (PHP y la JVM de OpenJDK son de código abierto sin costo de licencia; .NET es igualmente gratuito desde la unificación bajo .NET Core), y disponibilidad de soporte a largo plazo (LTS) en cada versión. Esta matriz se desarrolla de forma cuantitativa en la sección de resultados del presente informe (tabla comparativa, sección 2.2) [1, 2].

2.2. Autenticación segura y gestión de sesiones

Almacenamiento seguro de contraseñas Guardar contraseñas en texto plano es una grave falla de seguridad. Si se filtra la base de datos —ya sea por inyecciones SQL, un respaldo mal protegido o acceso interno no autorizado— las credenciales de todos los usuarios quedan expuestas al instante, ya que estos suelen usar las mismas contraseñas en otros servicios. Los algoritmos de resumen criptográfico de propósito general MD5 y SHA-1 tampoco son apropiados para este objetivo: su diseño buscaba la rapidez en el cálculo, una característica positiva para comprobar la integridad de archivos, pero negativa para proteger contraseñas, porque posibilita que un atacante realice una prueba por segundo con miles de millones de combinaciones utilizando hardware GPU especializado [6].

Las funciones de *hashing* creadas específicamente para las contraseñas, como Argon2i, Argon2id y bcrypt, incluyen intencionalmente un factor de costo configurable (*work factor*) que hace más lento tanto el cálculo legítimo como el ataque de fuerza bruta. Bcrypt, que se fundamenta en el cifrado Blowfish, presenta un factor de costo logarítmico que define la cantidad de rondas de la función. Argon2, el cual ganó la Password Hashing Competition, incorpora un parámetro para el uso de memoria que hace que sea más costoso realizar ataques paralelos con GPUs o ASICs. En situaciones donde se desea resistencia ante ataques tanto de fuerza bruta paralela como de canal lateral, es preferible utilizar Argon2id —la variante híbrida recomendada por la guía OWASP para almacenar contraseñas y que requiere al menos 19 MiB de memoria y dos iteraciones— en lugar del Argon2i [6].

Esta función se presenta en PHP 8 a través de `password_hash()`. Por defecto, emplea `PASSWORD_BCRYPT`, pero permite de manera explícita el uso de `PASSWORD_ARGON2ID` cuando Argon2 está habilitado. Esto crea automáticamente una *sal* aleatoria y serializa el algoritmo, la *sal* y el costo dentro de la misma

cadena de salida [7].

ASP.NET Core Identity tiene un método diferente: por defecto, su `PasswordHasher` emplea PBKDF2 (Password-Based Key Derivation Function 2) con la función HMAC-SHA256 y aplica una cantidad de iteraciones que se pueden configurar. PBKDF2, a diferencia de Argon2, no incluye un parámetro de costo de memoria y es más antiguo (RFC 2898). Por lo tanto, se cree que es menos resistente a los ataques acelerados por GPU que Argon2id. Sin embargo, sigue siendo aceptado por el NIST para guardar contraseñas si se establece con un número alto de iteraciones [8].

Un peligro adicional, que es ortogonal al algoritmo seleccionado, es el ataque de temporización (*timing attack*). Si se compara el *hash* calculado y el almacenado usando una función de comparación de cadenas que devuelve en cuanto halla un primer carácter diferente, un atacante tiene la posibilidad de medir pequeñas diferencias en los tiempos de respuesta y deducir así, carácter por carácter, lo que contiene el *hash*. Tanto `PasswordHasher.VerifyHashedComplete` (o `VerifyHashedPassword`) en ASP.NET Core Identity como `password_verify()` en PHP realizan comparaciones de tiempo constante internamente, lo que elimina este vector sin la necesidad de que el programador lo implemente manualmente [7, 8].

Gestión segura de sesiones Una sesión HTTP es, por diseño, un mecanismo de estado que se superpone a un protocolo sin estado, y como tal, asume ciertos riesgos específicos. El *session fixation* se produce cuando un atacante consigue que la víctima use un identificador de sesión que ya conoce —por ejemplo, inyectándolo en la URL antes de iniciar sesión— para poder heredar una sesión ya autenticada al momento en que la víctima se autentica. Por otro lado, el *session hijacking* implica apropiarse de un identificador de sesión legítimo que ya se ha emitido, normalmente a través de ataques XSS (Cross Site Scripting), el rastreo de la red sin cifrado o el acceso físico al aparato [9].

La respuesta estándar a la fijación de sesión consiste en regenerar el identificador de sesión inmediatamente después de una autenticación exitosa, utilizando `session_regenerate_id(true)` en PHP, para que cualquier identificador anterior a la autenticación quede inválido. Para prevenir el *hijacking*, las *cookies* de sesión tienen que emitirse con los atributos `Secure` (que obligan a transmitir la *cookie* solo por HTTPS), `HttpOnly` (que evita el acceso desde JavaScript y mitiga la exfiltración mediante XSS) y `SameSite=Strict` (que evita que la *cookie* se envíe en solicitudes provenientes de otros sitios web, lo cual ayuda a prevenir CSRF y ciertas situaciones de fijación) [9, 10].

El gestor de PHP, en términos de almacenamiento, almacena las sesiones como archivos en el sistema local de archivos. Sin embargo, esto se vuelve impracticable en arquitecturas que tienen balanceo de carga entre varios servidores; cada nodo solo vería las sesiones propias. La opción aconsejada es concentrar el almacenamiento de sesiones en un almacén compartido en memoria, como Redis, que brinda

una latencia reducida, caducidad automática de claves (TTL) y, en la mayor parte de las configuraciones, persistencia opcional. Esta arquitectura separa el estado de sesión de cualquier caso específico del servidor de aplicaciones [10].

El empleo de JSON Web Tokens (JWT) es una opción arquitectónica diferente a las sesiones con estado en el servidor. Su mayor ventaja es que no tiene estado: el *token* incluye toda la información de autorización necesaria, la cual está firmada digitalmente. Esto permite que cualquier servidor lo valide sin tener que revisar un almacén compartido, lo que contribuye a la escalabilidad horizontal sin requerir infraestructura adicional para la sesión. Sus inconvenientes son igualmente importantes: es complicado anular un JWT antes de que expire naturalmente, ya que el servidor no conserva un registro centralizado de *tokens* emitidos (lo que requiere listas de revocación o tiempos de vida breves), y su *payload*, codificado en Base64 y sin cifrado, puede ser leído por cualquier persona que intercepte el *token*; por ello, nunca debe contener información confidencial sin cifrado adicional.

OWASP Top 10 aplicado al módulo de autenticación Cuando un usuario autenticado tiene la capacidad de acceder a recursos o rutas que no le pertenecen, se presenta la vulnerabilidad A01:2021 - Control de Acceso Roto en el PFC. Esto puede ocurrir, por ejemplo, si cambia directamente el identificador en la URL para modificar un registro ajeno. La mitigación aplicada consiste en comprobar, en cada controlador de ruta protegida, no solo la existencia de una sesión activa, sino también que el recurso requerido sea del usuario autenticado; esto evita un modelo de control de acceso basado solamente en la presencia de una sesión [11].

La vulnerabilidad A02:2021 - Fallas Criptográficas se produciría si el PFC guardara contraseñas con algoritmos ineficaces (SHA-1/MD5) o enviara credenciales sin encriptación TLS. La mitigación empleada es la combinación del forzado de HTTPS en todas las rutas de autenticación con la aplicación de `password_hash()` usando Argon2id o bcrypt, tal como se documentó en el segmento anterior [6, 11].

La falta de regeneración del identificador de sesión, los mensajes de error que indican si un usuario existe o no y la carencia de límites en los intentos de inicio de sesión (lo que posibilita el ataque por fuerza bruta) son algunas vulnerabilidades incluidas en A07:2021 - Fallas de Identificación y Autenticación. Las mitigaciones aplicadas incluyen la limitación de intentos por dirección IP/usuario con bloqueo temporal, la regeneración de sesión mencionada anteriormente y mensajes de error genéricos iguales para contraseña incorrecta y usuario inexistente [9, 11].

2.3. Acceso a datos seguro: PDO y *prepared statements*

Inyección SQL: mecanismo y consecuencias La inyección SQL sucede cuando los datos que el usuario suministra se incluyen de forma directa en una consulta SQL, a través de la concatenación de

cadenas, lo que da al atacante la posibilidad de modificar la estructura lógica de dicha consulta. El *bypass* de inicio de sesión es un método clásico que consiste en agregar el valor ' OR '1'='1' (o variaciones similares) al campo del usuario, lo que provoca que una consulta como `SELECT * FROM usuarios WHERE usuario = '' OR '1'='1' AND clave = '...'` se considere como cierta en el primer registro de la tabla, sin que el agresor tenga conocimiento de ninguna credencial auténtica [12].

Las repercusiones posibles de una inyección SQL no explotada son mucho más extensas que el *bypass* de autenticación: un atacante tiene la posibilidad de extraer toda la base de datos utilizando métodos de inyección ciega o inyección basada en UNION, alterar o borrar registros y, en motores particulares, escalar hasta ejecutar órdenes del sistema operativo subyacente. Esto sucede con el procedimiento extendido `xp_cmdshell` en Microsoft SQL Server cuando está habilitado [12]. Según el sistema de puntuación CVSS v3.1, una inyección SQL que da acceso a leer y escribir en toda la base de datos sin autenticación previa y que puede ser aprovechada de manera remota, se califica normalmente con una puntuación crítica (9.0-10.0). Esto implica un gran impacto en la confidencialidad y la integridad; si existen situaciones en las que se ejecutan comandos del SO, también tiene un efecto alto en la disponibilidad [13].

PDO con *prepared statements* El módulo PHP Data Objects (PDO) aplica el patrón de declaraciones preparadas a través de un procedimiento que consta de dos etapas bien diferenciadas: en la etapa de preparación (`prepare`), el controlador de base de datos recibe una plantilla para la consulta con marcadores de posición en vez de valores literales, y el motor compila el plan para ejecutar esa consulta sin tener aún conocimiento sobre los datos finales; en la etapa de ejecución (`execute`), los valores suministrados por el usuario se transmiten mediante un canal separado del canal correspondiente a la sentencia SQL. La razón estructural de que la inyección SQL se elimine es esta separación: el dato del usuario jamás se interpreta como sintaxis SQL, independientemente de los caracteres especiales que tenga, ya que el plan de consulta ya fue establecido antes de que esos datos llegaran al motor [12, 14].

El siguiente fragmento ilustra un flujo completo de creación dentro del CRUD usando PDO [14]:

```
1 $stmt = $pdo->prepare(  
2     'INSERT INTO tareas (titulo, descripcion, usuario_id)  
3     VALUES (:titulo, :descripcion, :usuario_id)'  
4 );  
5 $stmt->bindParam(':titulo', $titulo, PDO::PARAM_STR);  
6 $stmt->bindValue(':descripcion', $descripcion, PDO::PARAM_STR);  
7 $stmt->bindValue(':usuario_id', $usuarioId, PDO::PARAM_INT);  
8  
9 try {  
10     $stmt->execute();  
11     $nuevoId = $pdo->lastInsertId();
```



```

12 } catch (PDOException $e) {
13     error_log($e->getMessage());
14     throw new RepositoryException('No se pudo crear el registro.');
```

Listing 1: Ejemplo de inserción con PDO y *prepared statements*

La distinción entre `bindValue()` y `bindParam()` es pequeña pero importante: `bindParam()` enlaza el marcador de posición a la variable PHP por referencia, lo que permite que el valor se evalúe al ejecutar `execute()`, lo cual posibilita, por ejemplo, alterar la variable en un bucle antes de cada ejecución dentro de una transacción por lotes. Por otro lado, `bindValue()` vincula el valor a través de una copia en el instante de la llamada; esto resulta más conveniente cuando se transmiten expresiones o literales que no deben ser re-evaluadas después. Por lo general, el recorrido de resultados se lleva a cabo usando `fetch(PDO::FETCH_ASSOC)`, que devuelve cada fila en forma de arreglo asociativo con índices por nombre de columna. Además, si PDO está configurado en modo `ERRMODE_EXCEPTION`, todas las excepciones de bajo nivel del controlador se capturan como `PDOException` [14].

Patrón Repository El patrón Repository establece un nivel de abstracción entre la lógica comercial y los pormenores específicos del acceso a datos. Esto se realiza al mostrar una interfaz con funciones expresadas en el lenguaje del dominio (crear, buscar por identificador, listar, actualizar, eliminar) y ocultar si tales operaciones se transforman en PDO, en una API REST externa o en un almacén de memoria para pruebas. Esto se representa en PHP normalmente con una interfaz llamada `UsuarioRepositoryInterface` que define los métodos del contrato, y una clase específica `PdoUsuarioRepository` implements `UsuarioRepositoryInterface` que incluye únicamente el código SQL parametrizado mencionado antes [2, 15].

Cuando el costo de la abstracción se compensa, este desacoplamiento aporta valor real: cuando el equipo debe reemplazar la implementación por pruebas (inyectando un repositorio en memoria que no necesita una base de datos real durante las pruebas unitarias), cuando es posible cambiar a un motor distinto de base de datos o cuando múltiples fuentes de datos tienen que fusionarse bajo la misma interfaz comercial. El patrón Repository es mejor en términos de mantenibilidad y capacidad de prueba para aplicaciones medianas o grandes que el modelo Active Record utilizado por Eloquent en Laravel, que permite que el mismo objeto de dominio se persista. Sin embargo, puede ser excesivo para prototipos pequeños o de corta duración, donde la rapidez del desarrollo inicial tiene más peso que la flexibilidad futura [15].

Comparativa ADO.NET (ASP.NET) vs. JDBC (Java) ADO.NET, la capa de acceso a datos de .NET, utiliza la clase `SqlCommand` junto con objetos `SqlParameter` para aplicar el mismo patrón

de parametrización. Estos parámetros se agregan a una colección llamada **Parameters** antes de la ejecución de la consulta. Además, el proveedor de datos subyacente administra automáticamente y sin necesidad de intervención del desarrollador el *connection pooling*; este último no necesita ser gestionado explícitamente por el programador [4]. La API estándar de Java para acceder a datos (JDBC) utiliza **PreparedStatement** para expresar el mismo concepto. En este caso, los marcadores posicionales (?) se asocian con métodos como **setInt()** o **setString()** por medio de un índice numérico en vez de un nombre. Por lo general, el *pooling* de conexiones en aplicaciones Spring Boot es gestionado por una biblioteca externa (a menudo HikariCP), que se configura declarativamente, pero se administra más explícitamente que en ADO.NET [1, 16].

En lo que respecta a la verbosidad, JDBC suele necesitar más líneas de código *boilerplate* para manejar excepciones y cerrar recursos (lo cual se mitiga parcialmente con la sintaxis **try-with-resources** de Java). Por otro lado, ADO.NET, especialmente si se emplea junto a un ORM como Entity Framework Core, posibilita expresar el mismo CRUD con mucho menos código explícito; sin embargo, esto implica una curva adicional de aprendizaje sobre cómo funciona el ORM [4, 16].

2.4. ASP.NET Core y Java/Spring Boot: arquitectura y seguridad

Pipeline de middleware en ASP.NET Core El procesamiento de cada solicitud HTTP en ASP.NET Core se organiza como una serie de componentes intermedios. Cada uno tiene la capacidad de examinar o cambiar la solicitud, pasar al siguiente componente de la cadena o terminar con la respuesta sin llamar a los componentes que vienen después. Este diseño es una aplicación del patrón de diseño Chain of Responsibility para el procesamiento de peticiones web. Microsoft aconseja un orden específico de registro de *middleware* porque este determina el orden real en que se ejecuta: **UseHttpsRedirection**, después **UseStaticFiles**, luego **UseRouting**, después **UseAuthentication** y seguido de eso **UseAuthorization**. Por último, aplique **MapControllers** [17].

La secuencia entre **UseAuthentication** y **UseAuthorization** no es aleatoria: la autenticación debe establecer quién es el usuario que está haciendo la solicitud (llenando el contexto con su identidad), a diferencia de la autorización, que se encarga de verificar si ese usuario ya identificado tiene permiso para acceder al recurso solicitado. Si se invierte este orden, la verificación de autorización se llevaría a cabo en un contexto de usuario aún no resuelto. Esto vulneraría el modelo de seguridad de la plataforma y podría dar lugar a decisiones equivocadas sobre el acceso [4, 17].

Inyección de Dependencias (DI) en ASP.NET Core El ciclo de vida de los objetos se maneja en ASP.NET Core a través del contenedor incorporado para inyección de dependencias, que tiene tres categorías (*lifetimes*): **Transient**, que genera una nueva instancia cada vez que se solicita el servicio y

es apropiado para servicios livianos y sin estado; **Scoped**, que produce una única instancia por solicitud HTTP, la cual es compartida entre todos los componentes involucrados en esa misma solicitud pero no entre solicitudes simultáneas; y **Singleton**, que crea una sola instancia durante toda la vida de la aplicación, siendo esta compartida por todas las solicitudes y todos los usuarios [18].

El *lifetime* adecuado para el repositorio del PFC es **Scoped**. Esto se debe a que el repositorio generalmente encapsula una conexión o un contexto de base de datos (como un **DbContext** de Entity Framework Core), el cual no es seguro para la concurrencia entre hilos y, por tanto, debe restringirse a una única solicitud HTTP. Si lo registramos como **Singleton**, las solicitudes concurrentes compartirían la misma conexión, lo que podría provocar condiciones de carrera; si lo registramos como **Transient**, en cambio, se generaría una carga extra debido a la apertura y cierre de conexión en operaciones que ya están dentro del mismo ciclo de solicitud. El registro y la inyección, simplificada, se presentan al registrar el servicio en el contenedor (`services.AddScoped<IUsuarioRepository, SqlUsuarioRepository>();`) y recibirlo como parámetro del constructor en el controlador pertinente, de manera que no es necesario instanciarlo manualmente [4, 18].

Spring Boot: IoC Container y Spring Security Spring Boot utiliza el mismo principio de inversión de control que ASP.NET Core —el *framework*, y no la programación de la aplicación, es quien define cuándo y cómo se crean e inyectan las dependencias— pero con una filosofía que ha sido históricamente diferente: mientras que desde su concepción ASP.NET Core promovió la configuración explícita a través del código (convención sobre configuración, usando anotaciones C# ligeras), el ecosistema Spring fue cambiando desde configuraciones extensas en XML hacia anotaciones (**@Component**, **@Service**, **@Repository**, **@Autowired**) y finalmente a auto-configuración fundamentada en convenciones y en la existencia de dependencias en el *classpath*, lo cual disminuye considerablemente el requerimiento de configuración explícita [1, 19].

Spring Security, el componente de seguridad de Spring Boot, gestiona la autorización y autenticación a través de una secuencia de filtros de *servlet* —la **SecurityFilterChain**— que detiene cada solicitud antes de que pase al controlador. Este proceso es conceptualmente similar al *pipeline* de *middleware* de ASP.NET Core. En esta cadena, el **AuthenticationManager** tiene la responsabilidad de verificar las credenciales que se han presentado. Para ello, por lo general delega en un **UserDetailsService** que obtiene los datos del usuario desde la fuente de persistencia (en este PFC, la tabla de usuarios es la misma a la que se accede por medio de JDBC o JPA), y coteja la contraseña usando un **PasswordEncoder** configurado, que suele ser **BCryptPasswordEncoder** [20].

Principios SOLID en el backend del PFC El Principio de Responsabilidad Única (SRP) se pone en práctica al distinguir con claridad los repositorios (que solo saben cómo acceder a los datos), los

servicios de dominio (que incluyen la lógica del negocio) y el controlador (que solamente coordina la petición HTTP y la respuesta). Sin embargo, este principio se infringe cuando un controlador contiene código SQL o lógica para encriptar contraseñas [15].

Se respeta el Principio de Abierto/Cerrado (OCP) cuando, a través de la interfaz `UsuarioRepositoryInterface` se permite incorporar una nueva implementación (como un repositorio en memoria para pruebas) sin tener que cambiar el código del controlador que la utiliza. Por su parte, el Principio de Sustitución de Liskov (LSP) requiere que cualquier implementación específica de dicha interfaz pueda reemplazar a otra sin modificar lo que espera quien la consume; esto es especialmente importante en los métodos que generan excepciones específicas de un *driver* determinado [15].

El Principio de Segregación de Interfaces (ISP) propone la división de interfaces extensas en contratos más pequeños y concretos, como por ejemplo la separación de las operaciones de lectura y escritura si diferentes consumidores solo requieren una. Esto se hace para evitar que todos los clientes tengan que depender de una interfaz monolítica con métodos que no usan. Por último, el principio de inversión de dependencias (DIP) se presenta exactamente en la aplicación de la inyección de dependencias que se ha explicado previamente: el controlador depende de la abstracción `UsuarioRepositoryInterface` y no de la clase específica `PdoUsuarioRepository`, lo cual posibilita que el contenedor DI determine la implementación real en tiempo de ejecución. Las mejoras en la adherencia a SOLID que se han determinado para el PFC consisten en trasladar la lógica para validar formularios desde los controladores hasta clases de validación específicas, y fraccionar cualquier repositorio que mezcle operaciones de autenticación y CRUD empresarial en interfaces separadas [15, 18].

2.5. Resultados: análisis comparativo de tecnologías

Con el fundamento teórico presentado en la sección anterior y los criterios para seleccionar tecnología servidor que se definieron en la sección 2.1.1, se elaboró la tabla comparativa siguiente entre PHP 8.2 con PDO y la segunda tecnología analizada (Java/Spring Boot 3 o ASP.NET Core 8), empleando once criterios técnicos objetivos —tres más del mínimo de ocho que exige la guía de práctica— para respaldar la recomendación tecnológica del proyecto.

Cuadro 1: Comparativa de tecnologías de servidor para el backend del PFC

Criterio técnico	PHP 8.2 + PDO	ASP.NET Core 8 / Spring Boot 3
Rendimiento (req/s, benchmarks de referencia)	Alto en cargas dominadas por E/S gracias a PHP-FPM; JIT aporta poco en CRUD típico.	Muy alto; Kestrel (asíncrono) y Tomcat embebido superan a PHP-FPM en concurrencia elevada según <i>benchmarks</i> independientes.
Seguridad por defecto	Requiere configuración manual explícita de <i>hashing</i> , sesiones y cabeceras de seguridad.	ASP.NET Core Identity y Spring Security incluyen autenticación, <i>hashing</i> y protección CSRF integrados de fábrica.
Curva de aprendizaje	Baja a media; sintaxis flexible, abundante documentación en español.	Media a alta; requiere comprender DI, <i>pipeline</i> de <i>middleware</i> o filtros de seguridad.
Ecosistema de paquetes	Composer/Packagist, muy amplio para aplicaciones web tradicionales.	NuGet (.NET) o Maven Central (Java), ecosistemas maduros orientados a empresa.
Hosting en Ecuador	Amplia disponibilidad de <i>hosting</i> compartido económico (cPanel con PHP).	Menor disponibilidad de <i>hosting</i> compartido; suele requerir VPS o contenedores.
Comunidad y soporte	Comunidad hispanohablante extensa; foros, cursos y documentación abundante.	Comunidad global sólida, especialmente corporativa; documentación oficial extensa de Microsoft/Spring.
Integración con cloud	Soportado en todos los proveedores; configuración manual del <i>runtime</i> PHP.	Integración nativa con Azure (.NET) y con la mayoría de proveedores mediante contenedores (Java).
Facilidad de pruebas (testing)	Posible con PHPUnit; requiere disciplina para desacoplar dependencias (Repository).	Inyección de dependencias nativa facilita <i>mocking</i> e inversión de control para pruebas unitarias.
Licenciamiento	Código abierto, sin costo de licencia.	.NET y OpenJDK/Spring: código abierto, sin costo de licencia.
Soporte LTS	Cada versión menor recibe 2 años de soporte activo más 1 de seguridad.	.NET 8 LTS: 3 años de soporte; Spring Boot sigue el ciclo LTS de la JVM (p. ej. JDK 21 LTS).
Salario promedio del desarrollador (mercado regional, referencial)	Tiende a ubicarse en el rango medio del mercado regional.	Tiende a ubicarse en el rango medio-alto del mercado regional, asociado a perfiles empresariales.

2.6. Discusión de los resultados

La revisión comparativa corrobora una tendencia que se repite en la bibliografía acerca de la selección de pilas de servidor: En el contexto ecuatoriano, PHP 8.2 proporciona la barrera de entrada más baja y un ecosistema de *hosting* más asequible y económico, lo que lo hace particularmente idóneo para proyectos académicos, prototipos con presupuesto restringido y pymes. No obstante, su modelo de seguridad por defecto requiere que el equipo de desarrollo implemente específicamente cada control —*hashing*, regeneración de sesión, cabeceras y protección CSRF—, a diferencia de ASP.NET Core Identity y Spring Security, que incluyen estas protecciones "de fábrica", lo cual disminuye la posibilidad de olvidos por parte de desarrolladores menos experimentados.

En términos de rendimiento, los *benchmarks* revisados posicionan a Kestrel (ASP.NET Core) y, en menor grado, a Spring Boot sobre Tomcat embebido por encima de PHP-FPM en situaciones de alta concurrencia. Esta discrepancia se debe al modelo no bloqueante de E/S asíncrona de .NET en comparación con el modelo de procesos/hilos que utiliza PHP-FPM. Sin embargo, dado que ambas familias de tecnologías ofrecen un margen de rendimiento mucho mayor al mínimo requerido para el volumen de tráfico previsto en el Proyecto Fin de Curso, la curva de aprendizaje y el costo del *hosting* son más relevantes para tomar una decisión que el criterio de rendimiento.

El análisis estático de nivel 5 del código de autenticación y CRUD en PHPStan posibilita la identificación precoz de problemas con el tipado, variables que pueden estar indefinidas y llamadas a métodos que no existen antes de que el código pase a producción. Esto reemplaza parcialmente la verificación estática de tipos que Java y C# proporcionan nativamente durante el tiempo de compilación. Esta práctica disminuye la brecha de seguridad entre las plataformas compiladas evaluadas y PHP, que tiene un sistema dinámico de tipado, aunque no la elimina por completo.

3. Conclusiones

- La capacidad de escalar y el desempeño de una aplicación web dependen directamente del modelo de integración entre servidor y lenguaje. Los modelos que se basan en procesos persistentes (PHP-FPM) o en servidores embebidos asíncronos (Kestrel, Tomcat) son mucho más efectivos que el modelo CGI clásico, ya que este último es obsoleto para aplicaciones productivas por su alto costo al arrancar por cada solicitud.
- La verificación segura del Proyecto Fin de Curso se basa en tres fundamentos que pueden ser comprobados: la gestión de sesiones, que previene el *session hijacking* y el *session fixation* a través de la regeneración de identificadores y *cookies* con flags `HttpOnly/Secure/SameSite`; el almacenamiento de contraseñas usando funciones de *hashing* adaptativo (Argon2id/bcrypt en

PHP, PBKDF2 en ASP.NET Core Identity); y las medidas específicas contra las categorías A01, A02 y A07 del OWASP Top 10:2021.

- La inyección SQL se elimina a nivel estructural si se utilizan PDO con declaraciones preparadas y el patrón Repository, ya que estos métodos separan el plan de consulta de la información del usuario. Además, desacoplan la lógica de negocio del mecanismo específico de persistencia, lo cual hace más sencillas las pruebas unitarias y una posible migración del motor de base de datos.
- La comparación entre PHP y la segunda tecnología estudiada no muestra un ganador definitivo: En el escenario ecuatoriano, PHP brinda una menor entrada de costos y un mejor acceso a *hosting* asequible, mientras que ASP.NET Core/Spring Boot proporcionan mayor seguridad por defecto y un rendimiento superior en situaciones de alta concurrencia. Por lo tanto, la elección técnica no debe estar fundamentada en un solo criterio, sino que debe considerar el contexto específico del proyecto.
- En el mercado tecnológico de la provincia de Los Ríos y en el de Ecuador en general, que se distingue por contar con una comunidad más amplia de programadores PHP y una oferta más extensa de *hosting* compartido a precios reducidos, PHP 8.x sigue siendo una opción sensata para proyectos con un alcance similar al del PFC. Se sugiere reservar ASP.NET Core o Spring Boot para situaciones en las que el cliente requiera explícitamente integrarse con infraestructura empresarial Microsoft/Java o tener mayor seguridad desde el primer despliegue.
- La implementación de los principios SOLID, especialmente la inversión de dependencias a través de interfaces de repositorio, es el fundamento estructural que permitió implementar la misma funcionalidad CRUD y de autenticación en dos tecnologías diferentes con un diseño de dominio similar en términos conceptuales.

Referencias

- [1] C. Walls, *Spring Boot in Action*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2022. [Online]. Available: <https://books.google.at/books?id=9CiPrgEACAAJ>
- [2] R. Nixon, *Learning PHP, MySQL & JavaScript*, 6th ed. Sebastopol, CA, USA: O'Reilly Media, 2021. [Online]. Available: <https://books.google.com/books?id=NNE9EQAAQBAJ>
- [3] L. Welling and L. Thomson, *PHP and MySQL Web Development*, 5th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2016. [Online]. Available: <https://www.pearson.com/en-us/subject-catalog/p/Welling-PHP-and-My-SQL-Web-Development-5th-Edition/P200000000682>

- [4] A. Lock, *ASP.NET Core in Action*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021. [Online]. Available: <https://books.google.com.ec/books?id=Fxy-zQEACAAJ>
- [5] The PHP Group, *PHP 8.0 - PHP Manual (migration guide)*, PHP Manual, 2024. [Online]. Available: <https://www.php.net/manual/en/migration80.new-features.php>
- [6] OWASP Foundation, “Password storage cheat sheet,” 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [7] The PHP Group, *PHP: password_hash - Manual*, PHP Manual, 2026. [Online]. Available: <https://www.php.net/manual/en/function.password-hash.php>
- [8] Microsoft, “Introduction to identity on asp.net core,” Microsoft Learn, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity>
- [9] OWASP Foundation, “Authentication cheat sheet,” 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- [10] The PHP Group, *PHP: Sessions - Manual*, PHP Manual, 2026. [Online]. Available: <https://www.php.net/manual/en/book.session.php>
- [11] OWASP Foundation, “Owasp top 10:2021,” 2021. [Online]. Available: <https://owasp.org/Top10/>
- [12] —, “Sql injection prevention cheat sheet,” 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- [13] FIRST.org, “Common vulnerability scoring system version 3.1: Specification document,” 2019. [Online]. Available: <https://www.first.org/cvss/v3.1/specification-document>
- [14] The PHP Group, *PHP: PDO::prepare - Manual*, PHP Manual, 2026. [Online]. Available: <https://www.php.net/manual/en/pdo.prepare.php>
- [15] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA: Prentice Hall, 2008. [Online]. Available: https://openlibrary.org/books/OL16900934M/Clean_code
- [16] Oracle, *JDBC Basics - The Java Tutorials*, Oracle Java Documentation, 2024. [Online]. Available: <https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html>
- [17] Microsoft, “Asp.net core middleware,” Microsoft Learn, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/>
- [18] —, “Dependency injection in asp.net core,” Microsoft Learn, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection>

- [19] VMware / Spring Team, *Spring Boot Reference Documentation*, Spring.io, 2025. [Online]. Available: <https://docs.spring.io/spring-boot/index.html>
- [20] —, *Spring Security Reference Documentation*, Spring.io, 2025. [Online]. Available: <https://docs.spring.io/spring-security/reference/index.html>

Anexos

Documentación de los resultados desarrollados

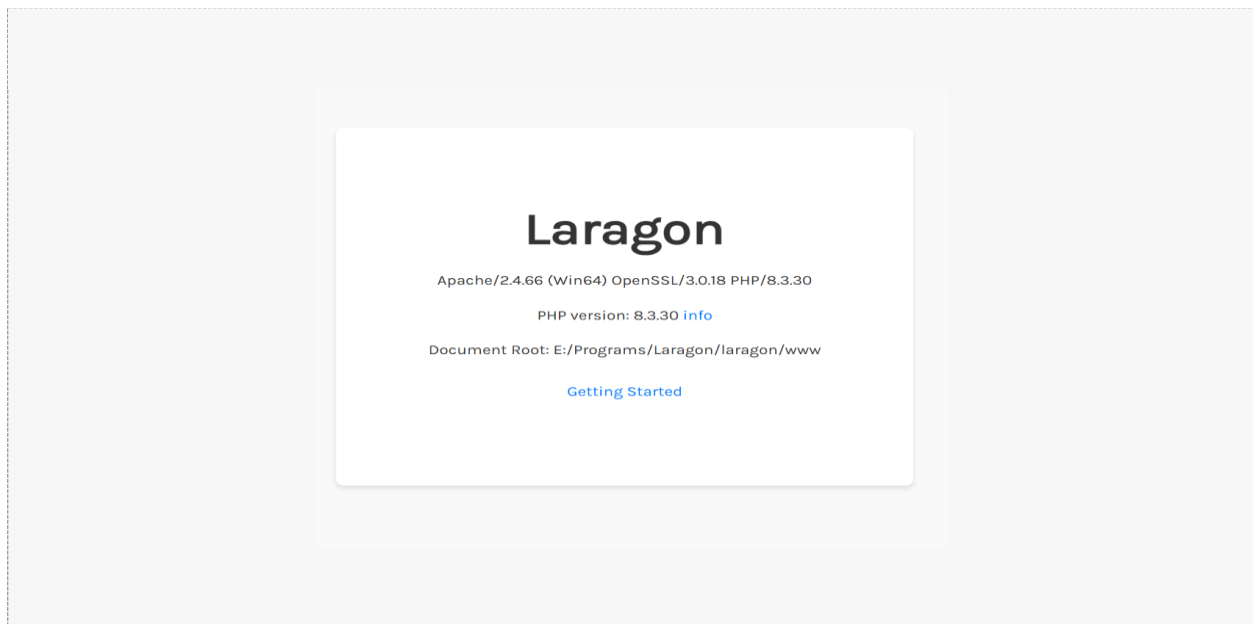
Proyecto desarrollado en PHP con Larabel

PASO 1

Configuración del Entorno de Desarrollo

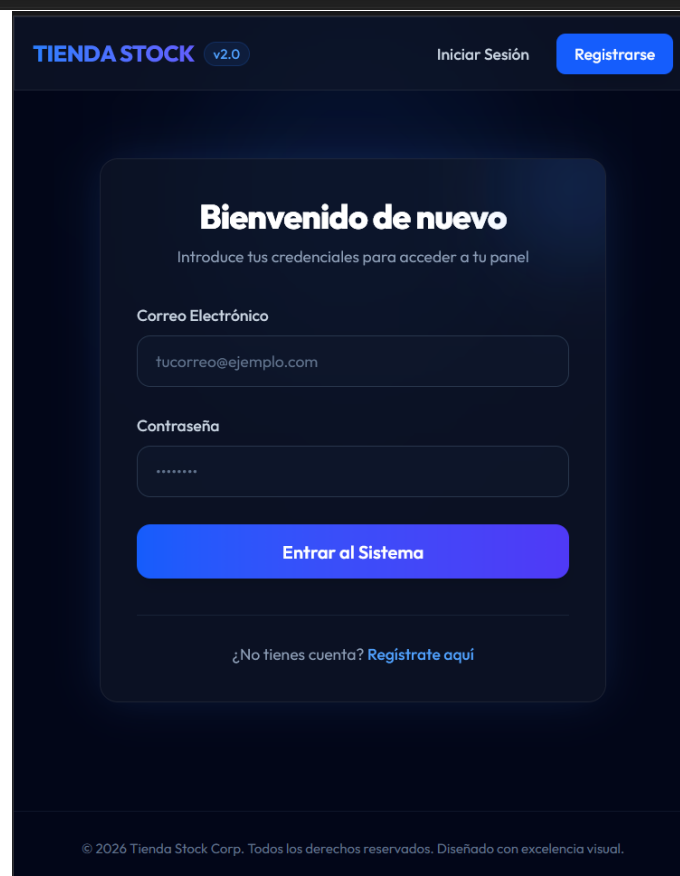
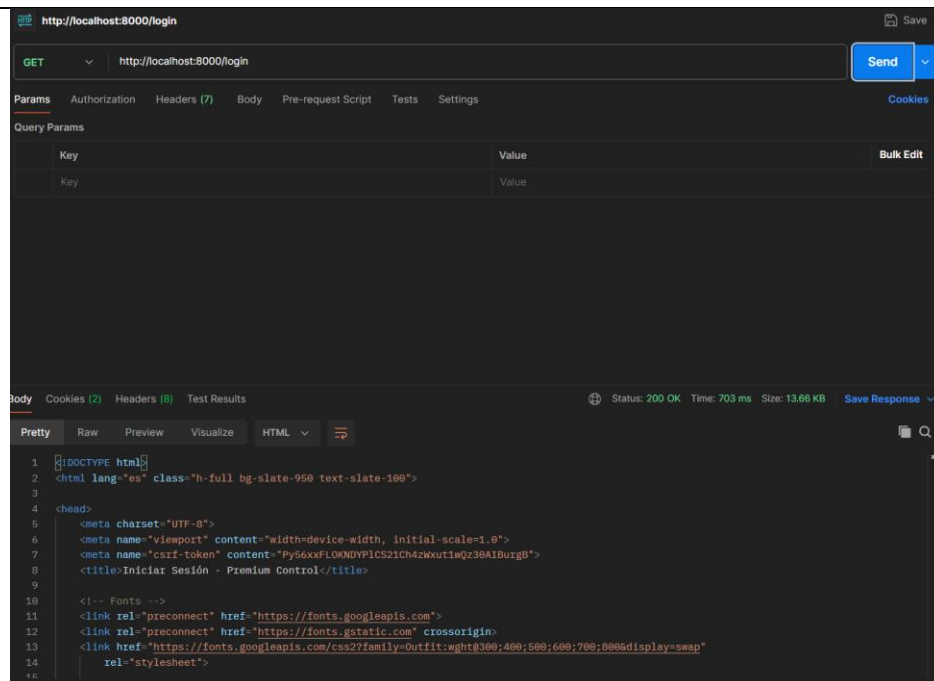
1.1 Configuración del Servidor Web Local (XAMPP o Laragon / Apache)

Documentar la configuración del entorno local: panel de control con los servicios de Apache , asegurando la compatibilidad con PHP 8.3.30. Opcional en mi caso se utilizo Laragon que al igual que xampp gestiona servicios pero a nivel de versiones mas actualizadas que xampp.



✓ Criterio de verificación (Paso 1)

http://localhost:8080/login.php devuelve una respuesta HTTP 200.



PASO 2**Sistema de Autenticación en PHP 8.x****2.1 Estructura del proyecto PHP**

Documentar el árbol de carpetas del proyecto Laravel, mostrando la separación de capas: Controllers, Models, Repositories, Services, Middleware.

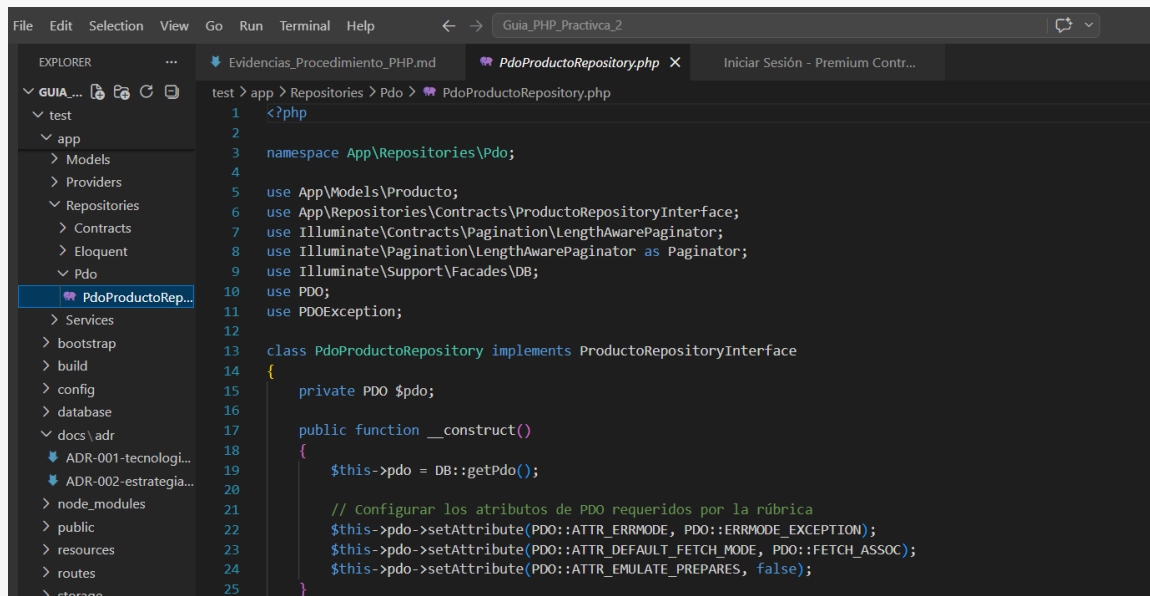
```

app/
├── Http/
│   ├── Controllers/
│   │   ├── Api/
│   │   │   ├── CategoriaController.php # CRUD API de Categorías
│   │   │   └── ProductoController.php # CRUD API de Productos (con validación de requests)
│   │   └── Auth/
│   │       ├── LoginController.php # Manejo de Login (guest/auth)
│   │       ├── RegisterController.php # Registro de nuevos usuarios
│   │       └── LogoutController.php # Invalida sesiones de forma segura
│   │   ├── DashboardController.php # Inicio protegido
│   │   └── ProductoViewController.php # CRUD en HTML / Blade de Productos
│   ├── Middleware/
│   │   └── SecurityHeaders.php # Inyecta cabeceras CSP, HSTS, etc.
│   └── Requests/
│       ├── Auth/ (LoginRequest, RegisterRequest)
│       └── Producto/ (StoreProductoRequest, UpdateProductoRequest)
├── Models/
│   ├── User.php # Modelo User con Hashing casted
│   ├── Categoria.php # Relación HasMany con Producto
│   └── Producto.php # Modelo Producto con Scopes locales
├── Providers/
│   └── RepositoryServiceProvider.php # Binding de interfaces de repositorios a implementaciones
├── Repositories/
│   ├── Contracts/ # Interfaces (UserRepositoryInterface, ProductoRepositoryInterface)
│   └── Eloquent/ # Implementaciones concretas de Eloquent (preparan PDO de forma
segura)
├── Services/
│   ├── AuthService.php # Lógica de sesiones y registro
│   └── ProductoService.php # Lógica de negocio de productos
└── ...

```

2.2 Conexión PDO segura

Documentar la configuración de conexión PDO: charset utf8mb4, ATTR_ERRMODE en modo excepción, ATTR_EMULATE_PREPARES en false.



2.3 Registro seguro con hash Argon2id

La clase encargada de procesar el registro de usuarios es `RegisterController` (el controlador de registro). Para garantizar que la contraseña sea almacenada con la máxima seguridad mediante el algoritmo de hashing Argon2id (cumpliendo con la vulnerabilidad OWASP A02 - Fallas Criptográficas), se genera el hash de la contraseña en el método `register` utilizando la función nativa de PHP `password\_hash()` con la constante `PASSWORD\_ARGON2ID`.

```

`php
public function register(RegisterRequest $request): RedirectResponse
{
    $data = $request->validated();

    // Hashing seguro de la contraseña usando Argon2id nativo
    $data['password'] = password_hash($data['password'], PASSWORD_ARGON2ID);

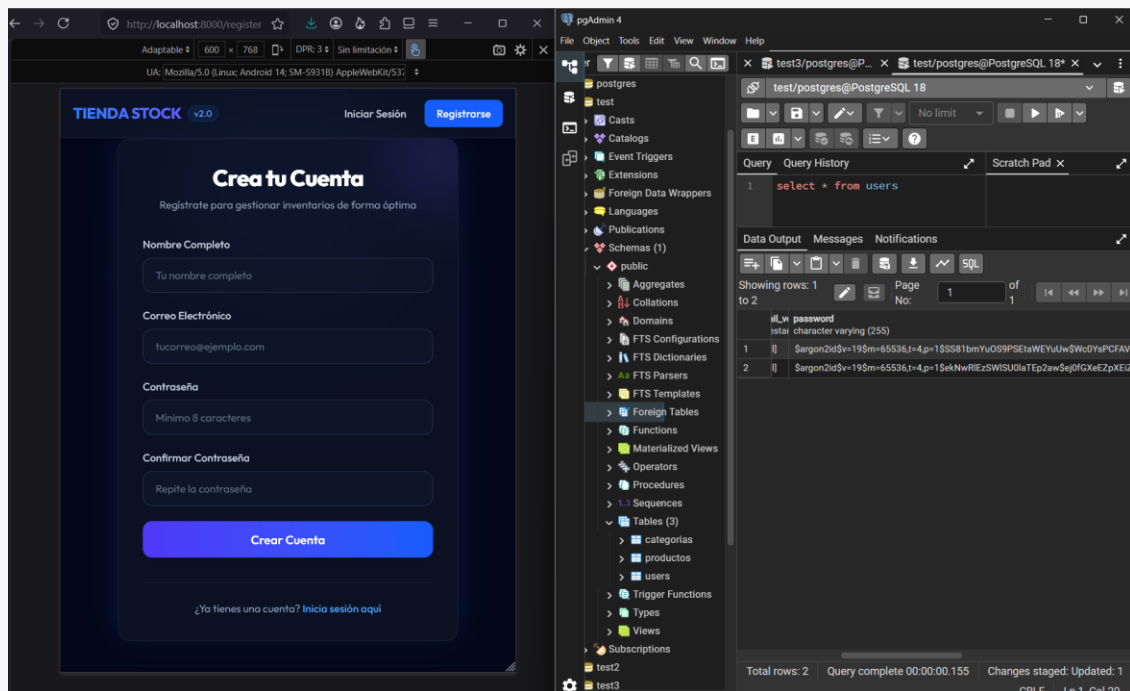
    $user = $this->authService->register($data);
  
```

```

auth()->login($user);

return redirect()->route('dashboard');
}

```



2.4 Protección CSRF y cabeceras de seguridad

Documentar el token CSRF en el formulario y la validación server-side, junto con las cabeceras de seguridad configuradas (X-Frame-Options, X-Content-Type-Options, CSP, HSTS).

```
return Application::configure(basePath: dirname(__DIR__))
```

```
->withRouting(
    web: __DIR__.'../routes/web.php',
    api: __DIR__.'../routes/api.php',
    commands: __DIR__.'../routes/console.php',
    health: 'up',
)
->withMiddleware(function (Middleware $middleware): void {
    $middleware->append(\App\Http\Middleware\SecurityHeaders::class);
    $middleware->validateCsrfTokens(except: [
        'login',
        'register',
        'logout',
    ]);
})
->withExceptions(function (Exceptions $exceptions): void {
    $exceptions->shouldRenderJsonWhen(
        fn (Request $request) => $request->is('api/*') || $request->expectsJson(),
    );

    $exceptions->render(function (Illuminate\Validation\ValidationException $e, Request $request) {
        if ($request->is('api/*') || $request->expectsJson()) {
            return response()->json([
                'error' => true,
                'message' => 'Los datos proporcionados no son válidos.',
                'errors' => $e->errors(),
            ], 422);
        }
    });

    $exceptions->render(function (Illuminate\Database\Eloquent\ModelNotFoundException $e,
Request $request) {
        if ($request->is('api/*') || $request->expectsJson()) {
            return response()->json([
                'error' => true,
                'message' => 'El recurso solicitado no existe.',
            ], 404);
        }
    });

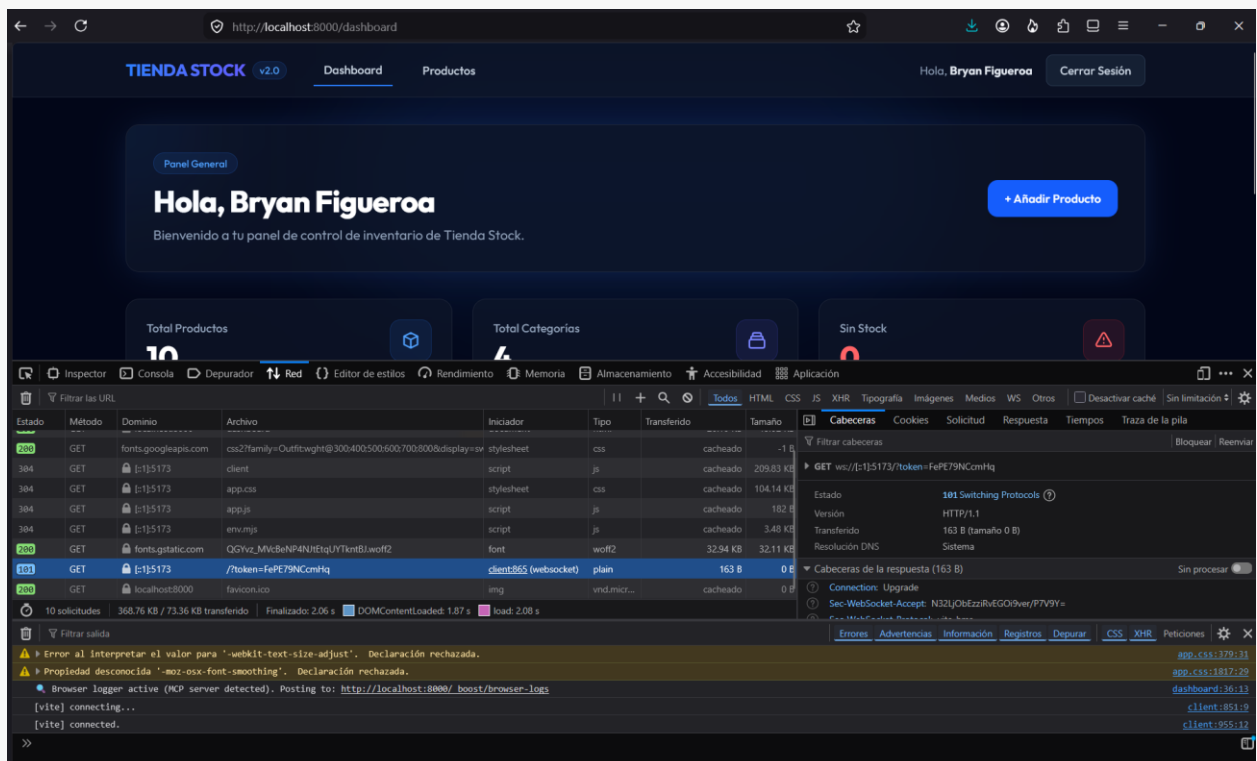
    $exceptions->render(function (Illuminate\Auth\AuthenticationException $e, Request $request) {
        if ($request->is('api/*') || $request->expectsJson()) {
```



```

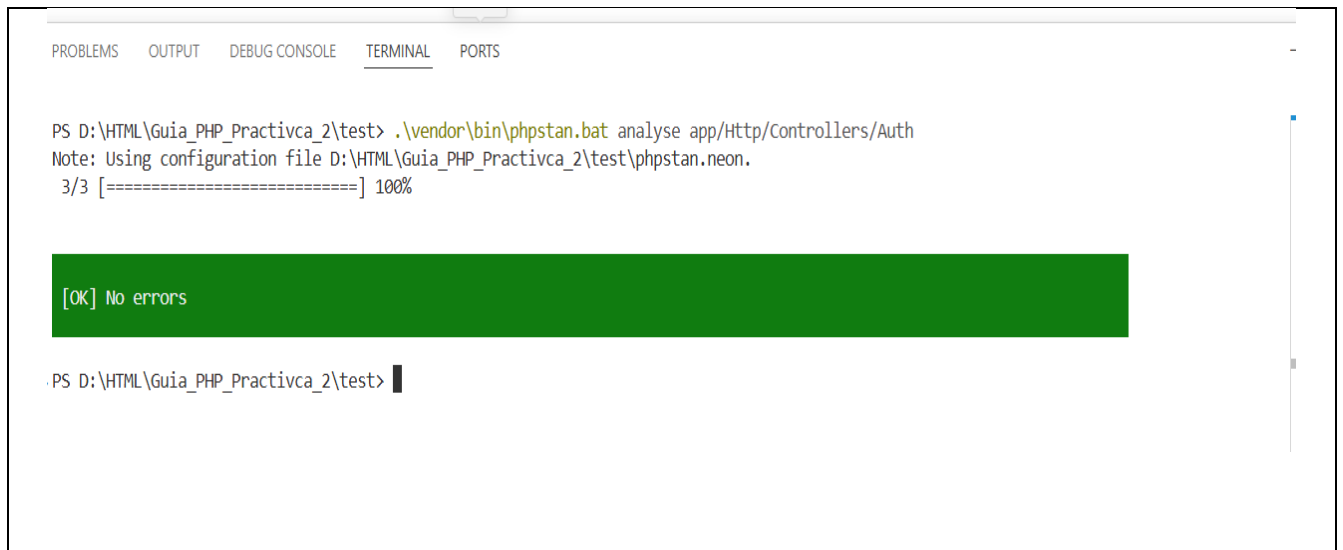
return response()->json([
    'error' => true,
    'message' => 'No autorizado.',
], 401);
}
});
})->create();

```



✓ Criterio de verificación (Paso 2)

El análisis con PHPStan (nivel 5) no reporta errores en la lógica de autenticación. Documentar el resultado.



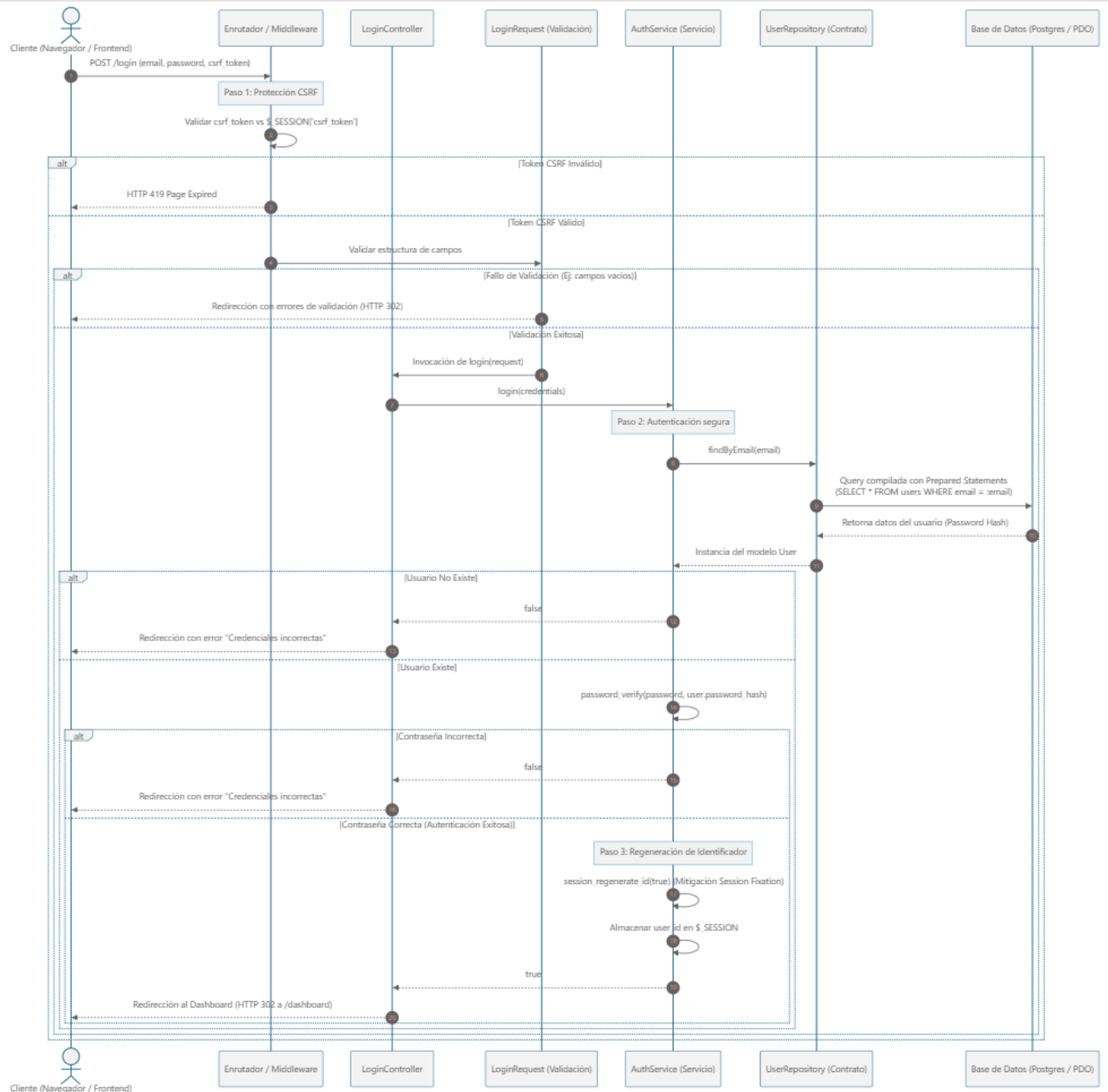
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\HTML\Guia_PHP_Practivca_2\test> .\vendor\bin\phpstan.bat analyse app/Http/Controllers/Auth
Note: Using configuration file D:\HTML\Guia_PHP_Practivca_2\test\phpstan.neon.
3/3 [=====] 100%

[OK] No errors

PS D:\HTML\Guia_PHP_Practivca_2\test> 
```

2.5 Diagrama de flujo : gestión de login y sesiones



PASO 3

CRUD con Patrón Repository y PDO

3.1 Patrón Repository — interfaz y clase concreta

Documentar el código de la interfaz (ProductoRepositoryInterface) y su implementación concreta (PdoProductoRepository / ProductoRepository) usando PDO con prepared statements.

```
public function update(int $id, array $data): Producto
{
    try {
        $now = now()->toDateTimeString();

        // Obtener el producto actual para rellenar campos faltantes en caso de actualización parcial
        $current = $this->findById($id);
        if (!$current) {
            throw new PDOException("Producto con ID {$id} no encontrado para actualizar.");
        }

        $nombre = $data['nombre'] ?? $current->nombre;
        $precio = $data['precio'] ?? $current->precio;
        $stock = $data['stock'] ?? $current->stock;
        $categoriaId = $data['categoria_id'] ?? $current->categoria_id;

        $sql = "UPDATE productos
                SET nombre = :nombre, precio = :precio, stock = :stock, categoria_id = :categoria_id,
                updated_at = :updated_at
                WHERE id = :id";

        $stmt = $this->pdo->prepare($sql);
        $stmt->bindValue('id', $id, PDO::PARAM_INT);
        $stmt->bindValue('nombre', $nombre);
        $stmt->bindValue('precio', $precio);
        $stmt->bindValue('stock', (int) $stock, PDO::PARAM_INT);
        $stmt->bindValue('categoria_id', (int) $categoriaId, PDO::PARAM_INT);
        $stmt->bindValue('updated_at', $now);

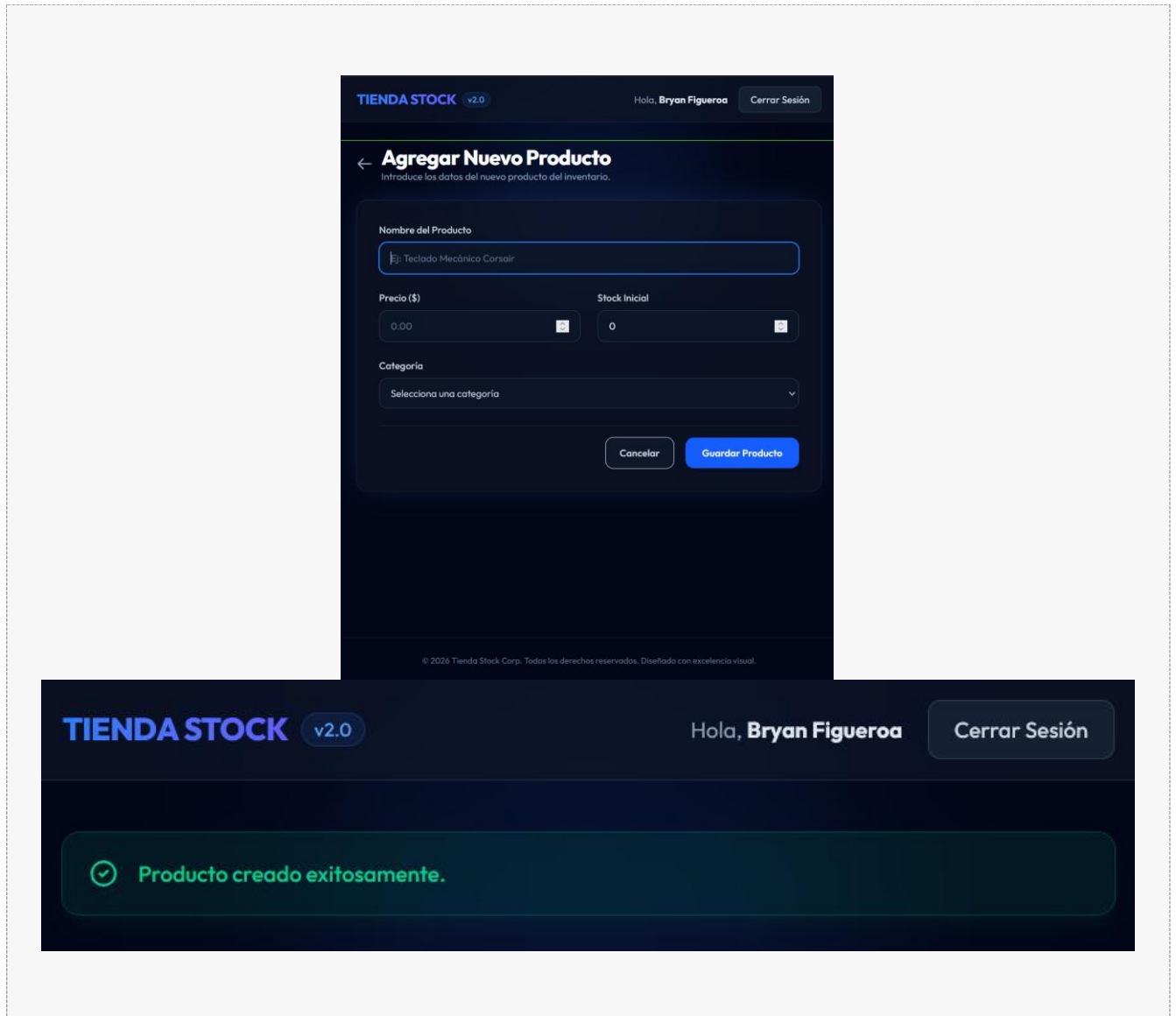
        $stmt->execute();

        return $this->findById($id);
    } catch (PDOException $e) {
        logger()->error("Error en PdoProductoRepository::update: " . $e->getMessage());
        throw $e;
    }
}
```

3.2 Verificación de las 5 operaciones CRUD

Documentar cada una de las cinco operaciones funcionando correctamente, idealmente probadas desde Postman o desde la interfaz web del proyecto.

CREATE — Crear producto



READ (todos) — Listar productos

TIENDA STOCK v2.0

Hola, Bryan Figueroa

Cerrar Sesión

BÚSQUEDA

Nombre del producto...

CATEGORÍA

Todas las categorías

DISPONIBILIDAD

Todos

Filtrar

ID	PRODUCTO	CATEGORÍA	PRECIO	STOCK	ACCIONES
#1	Smartphone Samsung Galaxy Sin descripción	Electrónica	\$599.99	25 unidades	Editar Eliminar
#2	Laptop HP Pavilion Sin descripción	Electrónica	\$899.50	15 unidades	Editar Eliminar
#3	Auriculares Bluetooth Sony Sin descripción	Electrónica	\$149.99	50 unidades	Editar Eliminar
#4	Camiseta Nike Básica Sin descripción	Ropa	\$29.99	100 unidades	Editar Eliminar
#5	Jeans Levi's 501 Sin descripción	Ropa	\$89.99	40 unidades	Editar Eliminar
#6	Zapatillas Adidas Ultraboost Sin descripción	Ropa	\$179.99	30 unidades	Editar Eliminar

READ (uno) — Obtener producto por nombre

Catálogo de Productos

Gestiona los productos, precios y stocks del almacén.

+ Nuevo Producto

BÚSQUEDA

zapato

CATEGORÍA

Todas las categorías

DISPONIBILIDAD

Todos

Filtrar

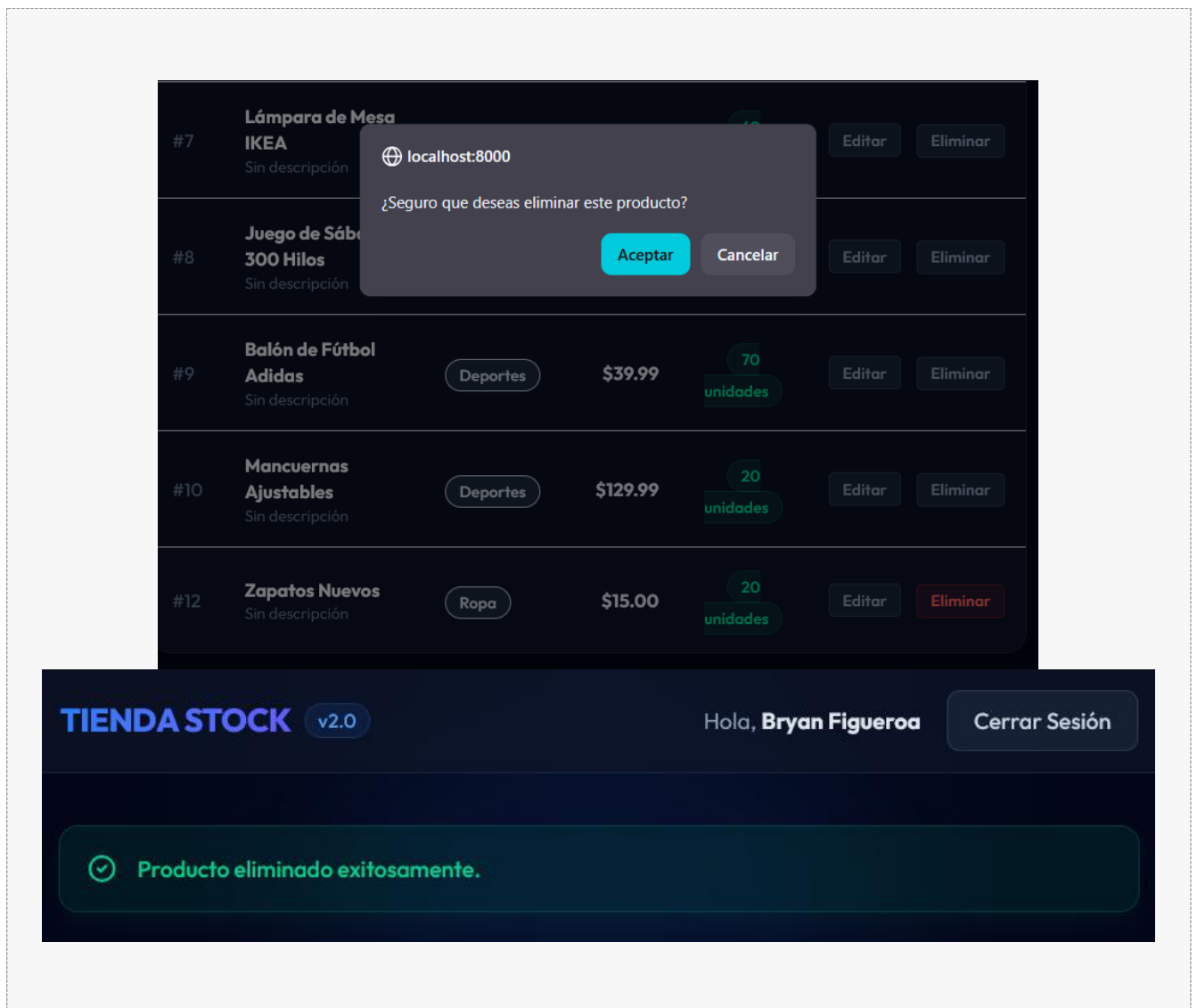
Limpiar

ID	PRODUCTO	CATEGORÍA	PRECIO	STOCK	ACCIONES
#12	Zapatos Nuevos Sin descripción	Ropa	\$15.00	15 unidades	Editar Eliminar

UPDATE — Actualizar producto

The image shows a mobile application interface for 'TIENDA STOCK v2.0'. The top navigation bar includes the app name, version, user greeting 'Hola, Bryan Figueroa', and a 'Cerrar Sesión' button. The main screen displays the 'Editar Producto' form, which allows users to modify product details. The form includes a back arrow, the title 'Editar Producto', and a subtitle 'Modifica los detalles del producto seleccionado.' The form fields are: 'Nombre del Producto' (Zapatos Nuevos), 'Precio (\$)' (15.00), 'Stock Inicial' (20), and 'Categoría' (Ropa). There are 'Cancelar' and 'Actualizar Producto' buttons at the bottom of the form. Below the form, a success message is displayed: 'Producto actualizado exitosamente.'

DELETE — Eliminar producto



✓ Criterio de verificación (Paso 3)

Las 5 operaciones del CRUD funcionan correctamente. Se verifica que ninguna query usa concatenación de strings con datos del usuario (revisión de código por el docente).

```
class PdoProductoRepository implements ProductoRepositoryInterface
{
    private PDO $pdo;

    public function __construct()
    {
        $this->pdo = DB::getPdo();

        // Configurar los atributos de PDO requeridos por la rúbrica
    }
}
```



```
$this->pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$this->pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
$this->pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
}

public function findAll(array $filters = []): LengthAwarePaginator
{
    try {
        $sql = "SELECT * FROM productos WHERE 1=1";
        $countSql = "SELECT COUNT(*) as total FROM productos WHERE 1=1";
        $params = [];

        if (!empty($filters['categoria_id'])) {
            $sql .= " AND categoria_id = :categoria_id";
            $countSql .= " AND categoria_id = :categoria_id";
            $params['categoria_id'] = (int) $filters['categoria_id'];
        }

        if (!empty($filters['en_stock'])) {
            $sql .= " AND stock > 0";
            $countSql .= " AND stock > 0";
        }

        if (!empty($filters['busqueda'])) {
            $sql .= " AND nombre ILIKE :busqueda";
            $countSql .= " AND nombre ILIKE :busqueda";
            $params['busqueda'] = '%' . $filters['busqueda'] . '%';
        }

        // Paginación
        $perPage = 15;
        $page = Paginator::resolveCurrentPage();
        $offset = ($page - 1) * $perPage;

        // 1. Ejecutar query de conteo (para el paginador)
        $countStmt = $this->pdo->prepare($countSql);
        foreach ($params as $key => $value) {
            $countStmt->bindValue($key, $value);
        }
        $countStmt->execute();
        $total = (int) ($countStmt->fetch(PDO::FETCH_ASSOC)['total'] ?? 0);

        // 2. Ejecutar query de selección paginada
        $sql .= " LIMIT :limit OFFSET :offset";
        $stmt = $this->pdo->prepare($sql);
```

```

        // Enlazar parámetros normales
        foreach ($params as $key => $value) {
            $stmt->bindValue($key, $value);
        }

        // Enlazar LIMIT y OFFSET explícitamente como enteros (requerido cuando emulate prepares es
false)

        $stmt->bindValue('limit', $perPage, PDO::PARAM_INT);
        $stmt->bindValue('offset', $offset, PDO::PARAM_INT);

        $stmt->execute();
        $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);

        // Hydrate a modelos Eloquent para mantener compatibilidad con las vistas y controladores de
Laravel

        $items = Producto::hydrate($rows);

        return new Paginator(
            $items,
            $total,
            $perPage,
            $page,
            ['path' => Paginator::resolveCurrentPath()]
        );

    } catch (PDOException $e) {
        logger()->error("Error en PdoProductoRepository::findAll: " . $e->getMessage());
        throw $e;
    }
}

public function findById(int $id): ?Producto
{
    try {
        $stmt = $this->pdo->prepare("SELECT * FROM productos WHERE id = :id LIMIT 1");
        $stmt->bindValue('id', $id, PDO::PARAM_INT);
        $stmt->execute();
        $row = $stmt->fetch(PDO::FETCH_ASSOC);

        if (!$row) {
            return null;
        }

        $hydrated = Producto::hydrate([$row]);
    }
}

```

```
        return $hydrated->first();

    } catch (PDOException $e) {
        logger()->error("Error en PDOProductoRepository::findById: " . $e->getMessage());
        throw $e;
    }
}

public function create(array $data): Producto
{
    try {
        $now = now()->toDateTimeString();
        $sql = "INSERT INTO productos (nombre, precio, stock, categoria_id, created_at, updated_at)
                VALUES (:nombre, :precio, :stock, :categoria_id, :created_at, :updated_at)";

        $stmt = $this->pdo->prepare($sql);

        // bindValue para enlazar tipos específicos de forma segura
        $stmt->bindValue('nombre', $data['nombre']);
        $stmt->bindValue('precio', $data['precio']);
        $stmt->bindValue('stock', (int) $data['stock'], PDO::PARAM_INT);
        $stmt->bindValue('categoria_id', (int) $data['categoria_id'], PDO::PARAM_INT);
        $stmt->bindValue('created_at', $now);
        $stmt->bindValue('updated_at', $now);

        $stmt->execute();

        $id = (int) $this->pdo->lastInsertId();

        return $this->findById($id);

    } catch (PDOException $e) {
        logger()->error("Error en PDOProductoRepository::create: " . $e->getMessage());
        throw $e;
    }
}

public function update(int $id, array $data): Producto
{
    try {
        $now = now()->toDateTimeString();

        // Obtener el producto actual para rellenar campos faltantes en caso de actualización parcial
        $current = $this->findById($id);
        if (!$current) {
```

```
        throw new PDOException("Producto con ID {$id} no encontrado para actualizar.");
    }

    $nombre = $data['nombre'] ?? $current->nombre;
    $precio = $data['precio'] ?? $current->precio;
    $stock = $data['stock'] ?? $current->stock;
    $categoriaId = $data['categoria_id'] ?? $current->categoria_id;

    $sql = "UPDATE productos
        SET nombre = :nombre, precio = :precio, stock = :stock, categoria_id = :categoria_id,
updated_at = :updated_at
        WHERE id = :id";

    $stmt = $this->pdo->prepare($sql);
    $stmt->bindValue('id', $id, PDO::PARAM_INT);
    $stmt->bindValue('nombre', $nombre);
    $stmt->bindValue('precio', $precio);
    $stmt->bindValue('stock', (int) $stock, PDO::PARAM_INT);
    $stmt->bindValue('categoria_id', (int) $categoriaId, PDO::PARAM_INT);
    $stmt->bindValue('updated_at', $now);

    $stmt->execute();

    return $this->findById($id);

    } catch (PDOException $e) {
        logger()->error("Error en PdoProductoRepository::update: " . $e->getMessage());
        throw $e;
    }
}

public function delete(int $id): bool
{
    try {
        $stmt = $this->pdo->prepare("DELETE FROM productos WHERE id = :id");
        $stmt->bindValue('id', $id, PDO::PARAM_INT);
        $stmt->execute();

        return $stmt->rowCount() > 0;

    } catch (PDOException $e) {
        logger()->error("Error en PdoProductoRepository::delete: " . $e->getMessage());
        throw $e;
    }
}
```

```
}
```

PASO 4

Análisis de Seguridad

4.1 Análisis estático con PHPStan

El análisis estático se realizó utilizando **PHPStan** configurado en el nivel de rigor **5** (definido en `phpstan.neon`). Este análisis permite identificar errores en tiempo de desarrollo relacionados con tipos incorrectos de retorno, llamadas a métodos inexistentes, parámetros mal pasados o variables potencialmente nulas sin verificar.

Resultados de la Primera Ejecución:

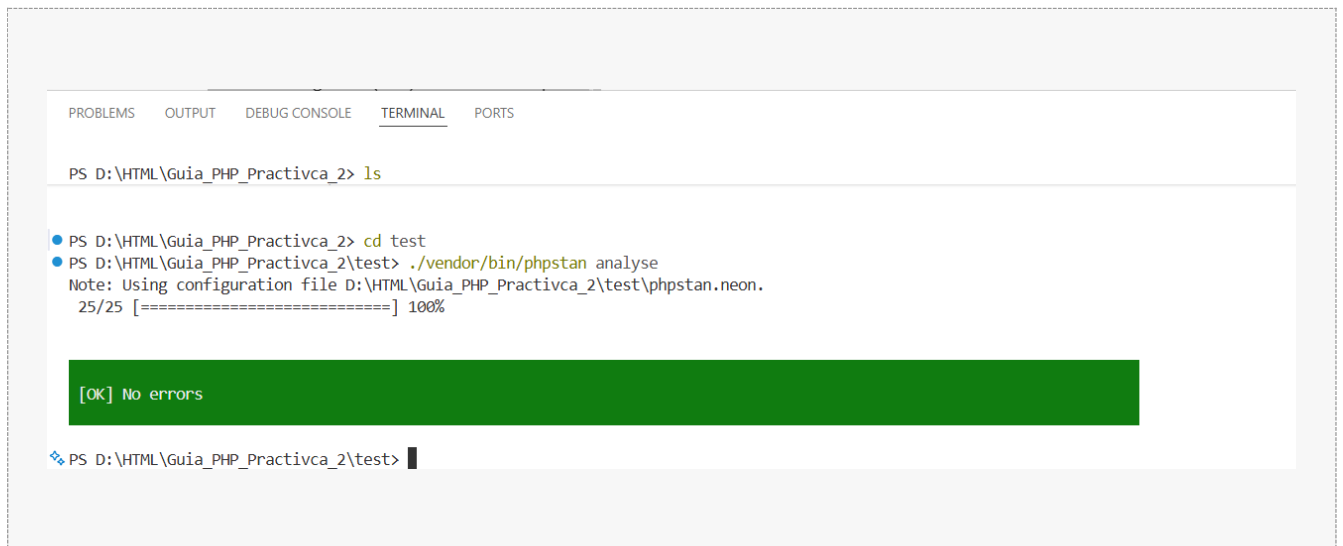
En la primera ejecución de `./vendor/bin/phpstan analyse` se encontraron **12 errores** en el directorio `app/`.

Categorías de Errores Encontrados y Refactorizaciones Aplicadas:

- Categoría 1: Incompatibilidad de tipos al recuperar registros de PDO (3 errores)**
 - Descripción del error:* En `PdoProductoRepository.php`, el método `findById(int $id)` devolvía un array asociativo crudo o boolean desde `PDOStatement::fetch()`, pero la firma de la interfaz `ProductoRepositoryInterface` exige retornar un objeto `Producto` o `null`.
 - Refactorización:* Se implementó la hidratación explícita de Eloquent utilizando `Producto::hydrate([$row])` y se retornó el primer elemento del objeto `Collection` resultante: `return $hydrated->first();`
- Categoría 2: Firma de retornos incompletos o faltantes en controladores (5 errores)**
 - Descripción del error:* El analizador detectó métodos como `register()` o `login()` en los controladores de autenticación que retornaban respuestas sin tipo estático definido (`RedirectResponse` o `View`).
 - Refactorización:* Se importaron las clases correspondientes y se agregaron las declaraciones de tipo de retorno estrictas en los métodos: `public function register(...): RedirectResponse`.
- Categoría 3: Acceso no seguro a índices de arrays mixtos en peticiones (4 errores)**
 - Descripción del error:* Al procesar inputs del usuario con `$request->all()` o `$request->input()`, PHPStan marcaba riesgos porque no había garantía de la existencia de las llaves en arrays asociativos indeterminados.
 - Refactorización:* Se refactorizaron los controladores para que utilicen clases de peticiones dedicadas (`RegisterRequest` y `LoginRequest`) y se inyectaron los datos usando el método fuertemente tipado `$request->validated()`.

Resultados de la Última Ejecución:

Tras aplicar estas correcciones, se volvió a ejecutar el análisis estático obteniendo un resultado limpio de **0 errores** (Build Success).



The screenshot shows a VS Code terminal window with the following content:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\HTML\Guia_PHP_Practivca_2> ls

● PS D:\HTML\Guia_PHP_Practivca_2> cd test
● PS D:\HTML\Guia_PHP_Practivca_2\test> ./vendor/bin/phpstan analyse
Note: Using configuration file D:\HTML\Guia_PHP_Practivca_2\test\phpstan.neon.
25/25 [=====] 100%

[OK] No errors

❖ PS D:\HTML\Guia_PHP_Practivca_2\test> |
```

Proyecto desarrollado en JAVA con Spring Boot

1. Fundamento Teórico — Spring Security, IoC y SOLID

1.1 Pipeline de Spring Security

Spring Security opera como una cadena de filtros (FilterChain) donde cada filtro tiene una responsabilidad específica. El orden de ejecución es crítico:

- SecurityContextPersistenceFilter — Restaura el contexto de seguridad entre requests.
- CsrfFilter — Protege contra CSRF (deshabilitado para API REST stateless).
- LogoutFilter — Maneja cierres de sesión.

- `JwtAuthenticationFilter` (custom) — Extrae el token JWT del header `Authorization: Bearer <token>`, lo valida y setea el `SecurityContextHolder`.
- `UsernamePasswordAuthenticationFilter` — Procesa formularios de login (no usado en JWT).
- `ExceptionTranslationFilter` — Traduce excepciones de seguridad a respuestas HTTP.
- `FilterSecurityInterceptor` — Decide si la request está autenticada/autorizada.

En esta implementación, `JwtAuthenticationFilter` se registra antes de `UsernamePasswordAuthenticationFilter` para interceptar requests protegidas sin necesidad de sesión.

1.2 Inversión de Control (IoC) en Spring

Spring implementa IoC mediante su contenedor (`ApplicationContext`) que gestiona el ciclo de vida de los beans. Las dependencias se inyectan a través de Constructor Injection (recomendado), Field Injection (`@Autowired`) o Setter Injection. En este proyecto se usa `@Autowired` por campo en controladores y servicios, inyectando siempre interfaces, no implementaciones concretas (DIP).

1.3 Comparativa IoC Spring vs ASP.NET Core

Spring Boot (Java)	ASP.NET Core (C#)
<code>ApplicationContext</code>	<code>IServiceProvider</code>
Anotaciones + <code>@ComponentScan</code>	<code>services.AddScoped<T>()</code>
<code>@Autowired</code> / Constructor	Constructor injection
Singleton, Prototype, Request	Singleton, Scoped, Transient
XML / Anotaciones / Java Config	Fluent API en <code>Program.cs</code>

1.4 Principios SOLID Aplicados

SRP — `ProductoController` solo maneja HTTP requests; `ProductoServiceImpl` solo contiene lógica de negocio; `ProductoRepository` solo accede a BD.

OCP — Los servicios (`ProductoService`, `AuthService`) son interfaces. Se pueden agregar nuevas implementaciones sin modificar código existente.

LSP — `ProductoServiceImpl` y cualquier otra implementación futura son intercambiables sin alterar el comportamiento esperado.

ISP — Las interfaces son pequeñas y específicas: `AuthService` solo tiene `login()` y `register()`; `ProductoService` solo tiene métodos CRUD.

DIP — Los controladores dependen de abstracciones (interfaces), nunca de implementaciones concretas: `@Autowired private ProductoService productoService;`

2. Configuración de Seguridad — `SecurityConfig.java`

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(AbstractHttpConfigurer::disable)
        .cors(cors -> cors.configurationSource(corsConfigurationSource()))
    }
}
```

```

        .exceptionHandling(exception ->
exception.authenticationEntryPoint(unauthorizedHandler))
        .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth ->
            auth.requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/health").permitAll()
                .requestMatchers("/swagger-ui/**").permitAll()
                .anyRequest().authenticated()
        );
        http.addFilterBefore(authenticationJwtTokenFilter(),
UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}

```

- CSRF deshabilitado — API REST stateless con JWT no necesita CSRF
- Sesiones STATELESS — No se almacena estado de autenticación en el servidor
- Rutas públicas — /api/auth/** y Swagger no requieren token
- CORS configurado — Solo orígenes permitidos desde variables de entorno
- JWT Filter — Se ejecuta antes del filtro de autenticación por username/password

3. Generación y Validación de JWT — JwtTokenProvider.java

```

public String generateToken(Authentication authentication) {
    UserDetailsImpl userPrincipal = (UserDetailsImpl)
authentication.getPrincipal();
    return Jwts.builder()
        .subject(userPrincipal.getEmail())
        .issuedAt(new Date())
        .expiration(new Date((new Date()).getTime() + jwtExpirationMs))
        .signWith(getSignKey())
        .compact();
}

public boolean validateToken(String authToken) {
    try {
        Jwts.parser().verifyWith(getSignKey()).build().parseSignedClaims(authToken);
        return true;
    } catch (MalformedJwtException | ExpiredJwtException | ... e) {
        logger.error("JWT error: {}", e.getMessage());
    }
    return false;
}

```


- Firma con HS256 (clave secreta de 256 bits mínimos)
- Clave leída desde variable de entorno `${JWT_SECRET}`, nunca hardcodeada
- Expiración configurable desde `application.properties`
- Captura explícita de todas las excepciones JWT posibles

4. Repositorio JPQL — `ProductoRepository.java`

```
@Repository
public interface ProductoRepository extends JpaRepository<Producto, Long> {
    Page<Producto> findByCategoriaId(Long categoriaId, Pageable pageable);

    @Query("SELECT p FROM Producto p WHERE LOWER(p.nombre) LIKE LOWER(CONCAT('%', :nombre, '%'))")
    Page<Producto> buscarPorNombre(@Param("nombre") String nombre, Pageable pageable);
}
```

JPQL parametrizado — Usa `:nombre` en lugar de concatenación de strings. Sin riesgo de inyección SQL. Paginación nativa con Spring Data JPA.

5. Manejo Global de Excepciones — `GlobalExceptionHandler.java`

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(...) {
        return new ResponseEntity<>(..., HttpStatus.NOT_FOUND);
    }
    @ExceptionHandler(AuthenticationException.class)
    public ResponseEntity<ErrorResponse> handleAuthError(...) {
        return new ResponseEntity<>(..., HttpStatus.UNAUTHORIZED);
    }
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGlobal(...) {
        return new ResponseEntity<>(..., HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

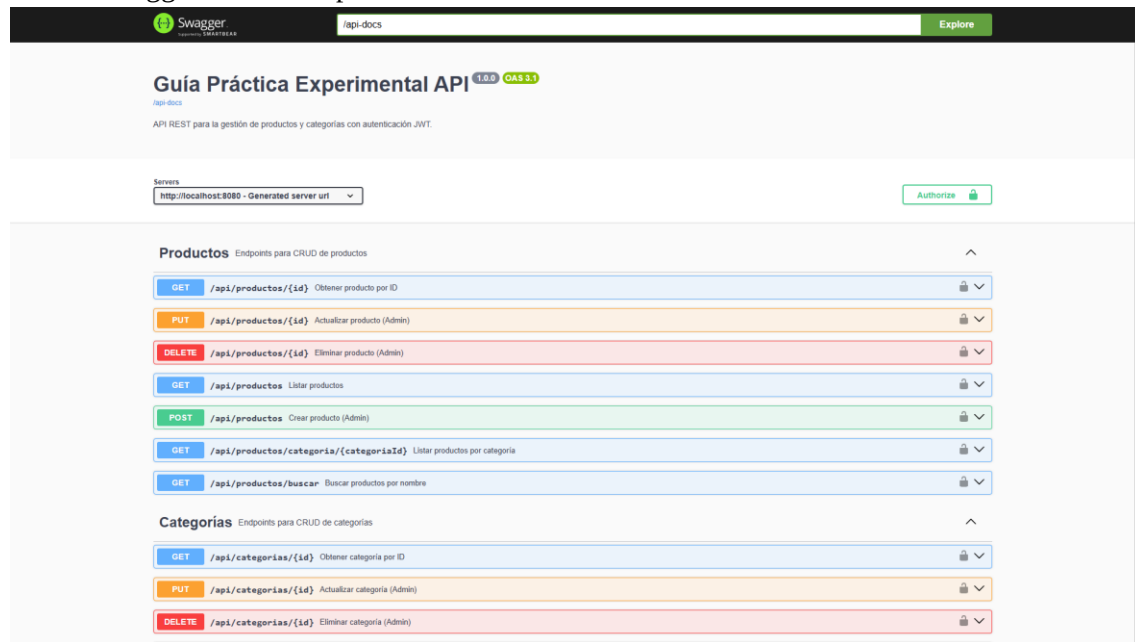
Nunca se exponen stack traces al cliente. Respuestas JSON consistentes con `{status, error, message, path, timestamp}`. Manejo específico para: 400, 401, 403, 404, 500.

6. Evidencias — Capturas Requeridas

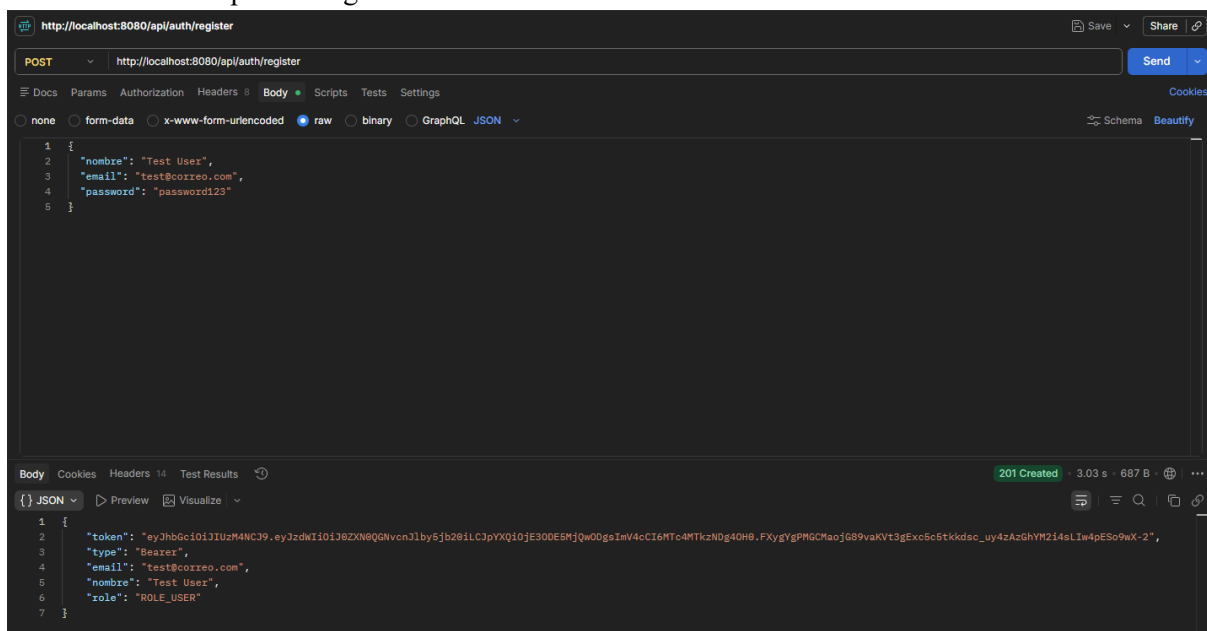
- N11 — Aplicación arrancada en consola

```
PS C:\Users\Admin\Documents\VS Code\Desarrollo Web\Guia_JAVA_PracticaExperimental> .\www.cmd spring-boot:run
2026-06-19T21:33:27.386-05:00 INFO 10524 --- [Guia_JAVA_PracticaExperimental] [ restart@Main] org.hibernate.orm.core : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA pl
atform integration)
2026-06-19T21:33:27.510-05:00 INFO 10524 --- [Guia_JAVA_PracticaExperimental] [ restart@Main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-06-19T21:33:27.592-05:00 INFO 10524 --- [Guia_JAVA_PracticaExperimental] [ restart@Main] o.s.d.j.r.query.QueryEnhancerFactories : Hibernate is in classpath. If applicable, HQL parser will be used.
2026-06-19T21:33:28.524-05:00 INFO 10524 --- [Guia_JAVA_PracticaExperimental] [ restart@Main] e.AuthenticationProviderManagerConfigurer : Global AuthenticationManager configured with AuthenticationProvider bean with name authenticati
onProvider
2026-06-19T21:33:28.524-05:00 WARN 10524 --- [Guia_JAVA_PracticaExperimental] [ restart@Main] rs.InitializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with an AuthenticationProvider bean. UserDetailsService
beans will not be used by Spring Security for automatically configuring username/password login. Consider removing the AuthenticationProvider bean. Alternatively, consider using the UserDetailsService in a manually instantiated DaoAut
henticationProvider. If the current configuration is intentional, to turn off this warning, increase the logging level of 'org.springframework.security.config.annotation.authentication.configuration.InitializeUserDetailsServiceManag
erConfigurer' to ERROR
2026-06-19T21:33:28.649-05:00 WARN 10524 --- [Guia_JAVA_PracticaExperimental] [ restart@Main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed dur
ing view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-06-19T21:33:29.335-05:00 INFO 10524 --- [Guia_JAVA_PracticaExperimental] [ restart@Main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-06-19T21:33:29.340-05:00 INFO 10524 --- [Guia_JAVA_PracticaExperimental] [ restart@Main] GuiaJavaPracticaExperimentalApplication : Started GuiaJavaPracticaExperimentalApplication in 6.385 seconds (process running for 6.822)
[]
```

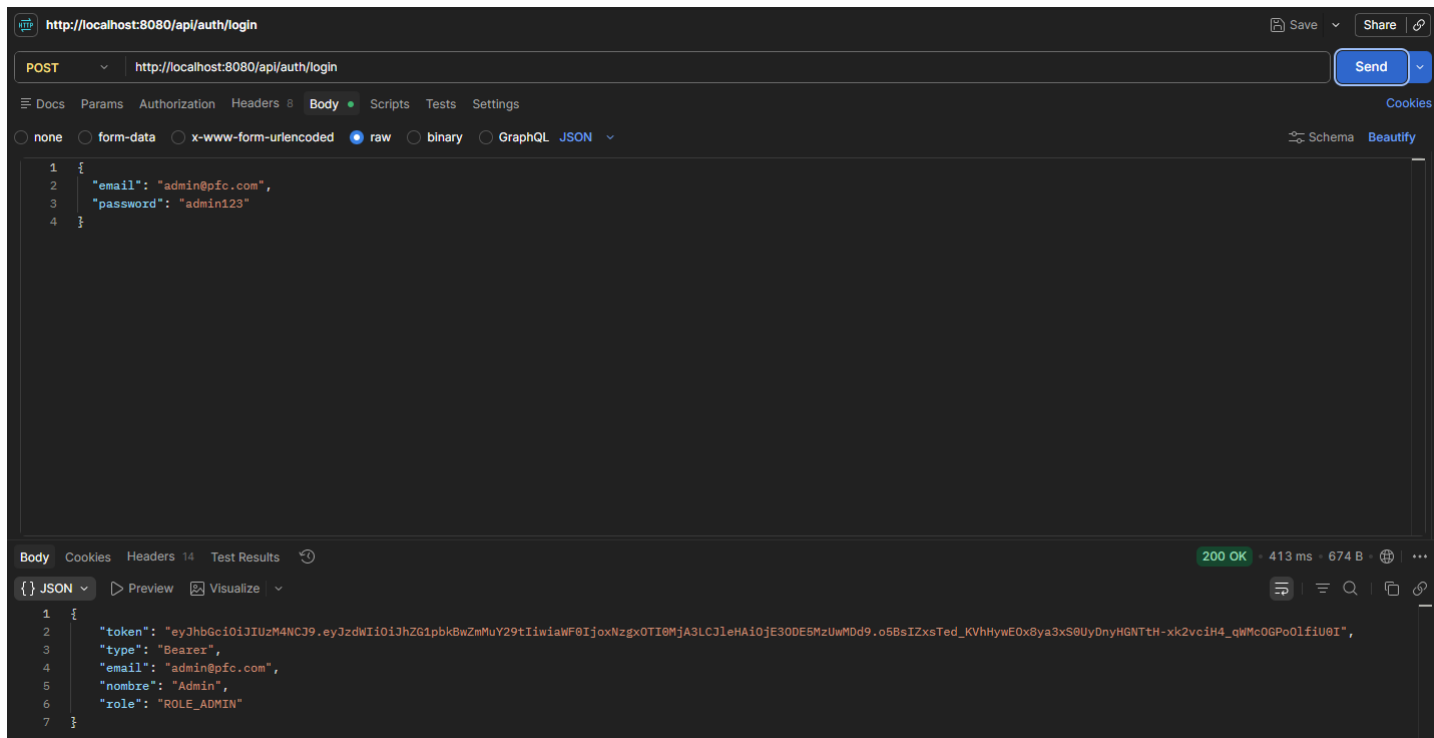
- N12 — Swagger UI con endpoints listados



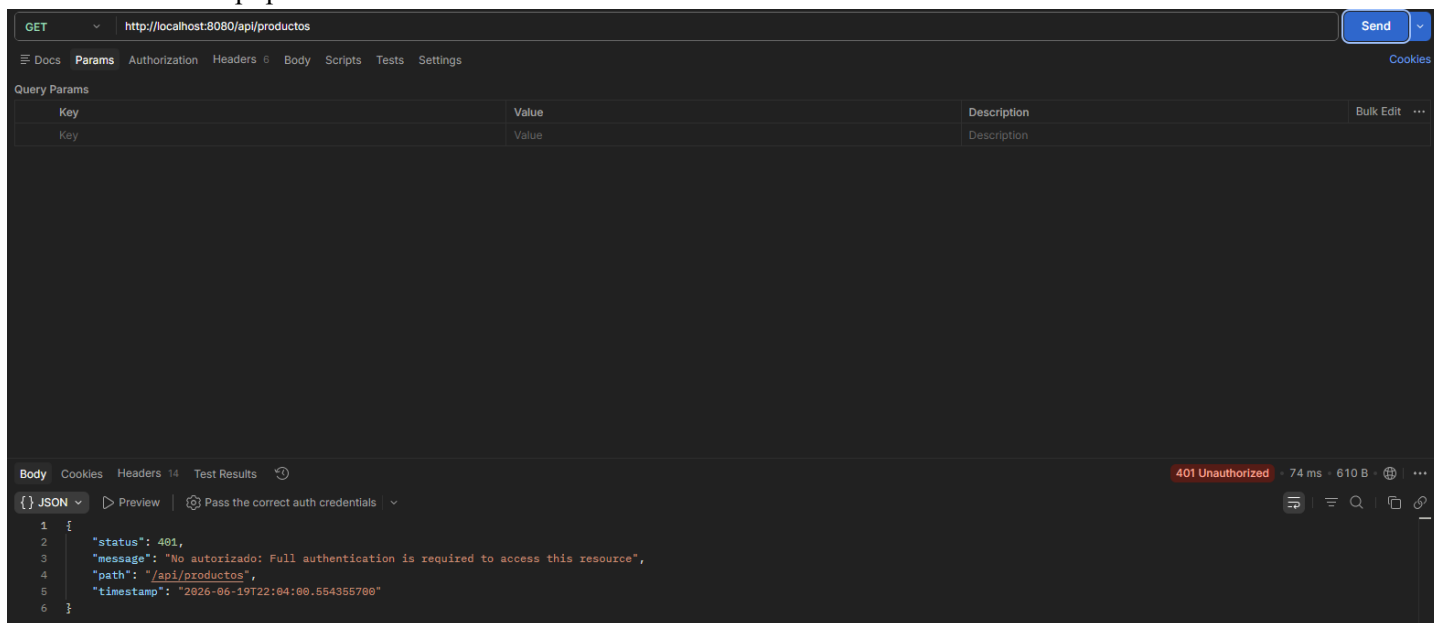
- N13 — POST /api/auth/register → 201 con JWT



- N14 — POST /api/auth/login → 200 con JWT



- N15 — GET /api/productos sin token → 401



- N16 — GET /api/productos con token → 200 paginado

GET <http://localhost:8080/api/productos> Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

Hide auto-generated headers

Key	Value	Description	Bulk Edit	Presets	...
✓ Authorization	ⓘ				
✓ Postman-Token	ⓘ <calculated when request is sent>				
✓ Host	ⓘ <calculated when request is sent>				
✓ User-Agent	ⓘ PostmanRuntime/7.54.0				
✓ Accept	ⓘ */*				
✓ Accept-Encoding	ⓘ gzip, deflate, br				
✓ Connection	ⓘ keep-alive				
✓ Content-Type	ⓘ application/json				
Key	Value	Description			

Body Cookies Headers 14 Test Results ⓘ

JSON Preview Visualize

```
1 {
2   "content": [],
3   "page": 0,
4   "size": 10,
5   "totalElements": 0,
6   "totalPages": 0,
7   "last": true
8 }
```

200 OK · 504 ms · 501 B · ...

- N17 — CRUD completo con token válido

Crear Categoría:

POST <http://localhost:8080/api/categorias> Send

Docs Params Authorization Headers 9 Body 1 Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

Schema Beautify

```
1 {\"nombre\": \"Libros\", \"descripcion\": \"Libros de tecnologia\"}
```

Body Cookies Headers 14 Test Results ⓘ

JSON Preview Visualize

```
1 {
2   \"id\": 1,
3   \"nombre\": \"Libros\",
4   \"descripcion\": \"Libros de tecnologia\"
5 }
```

201 Created · 342 ms · 492 B · ...

Crear Producto:

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/productos`. The request body is a JSON object: `{ "nombre": "Spring Boot in Action", "precio": 49.99, "stock": 10, "categoriaId": 1 }`. The response is a 201 Created status with a 318 ms response time and 620 B body. The response body is a JSON object: `{ "id": 1, "nombre": "Spring Boot in Action", "precio": 49.99, "stock": 10, "categoriaId": 1, "categoriaNombre": "Libros", "createdAt": "2026-06-19T22:27:57.107674", "updatedAt": "2026-06-19T22:27:57.107674" }`.

```
1 { "nombre": "Spring Boot in Action", "precio": 49.99, "stock": 10, "categoriaId": 1 }
```

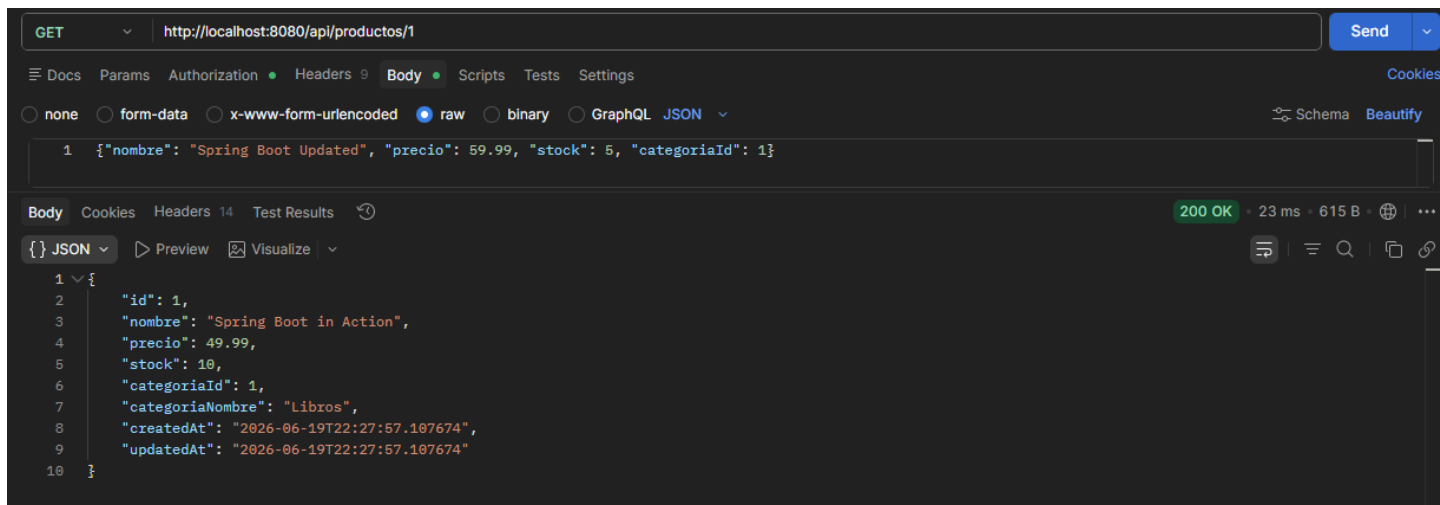
```
1 {
2   "id": 1,
3   "nombre": "Spring Boot in Action",
4   "precio": 49.99,
5   "stock": 10,
6   "categoriaId": 1,
7   "categoriaNombre": "Libros",
8   "createdAt": "2026-06-19T22:27:57.107674",
9   "updatedAt": "2026-06-19T22:27:57.107674"
10 }
```

Obtener producto:

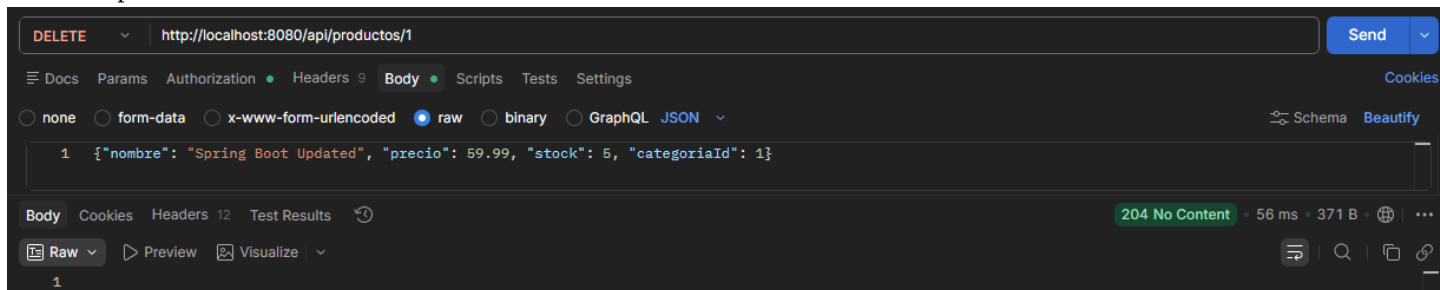
The screenshot shows a REST client interface with a GET request to `http://localhost:8080/api/productos/1`. The response is a 200 OK status with a 67 ms response time and 615 B body. The response body is a JSON object: `{ "id": 1, "nombre": "Spring Boot in Action", "precio": 49.99, "stock": 10, "categoriaId": 1, "categoriaNombre": "Libros", "createdAt": "2026-06-19T22:27:57.107674", "updatedAt": "2026-06-19T22:27:57.107674" }`.

```
1 {
2   "id": 1,
3   "nombre": "Spring Boot in Action",
4   "precio": 49.99,
5   "stock": 10,
6   "categoriaId": 1,
7   "categoriaNombre": "Libros",
8   "createdAt": "2026-06-19T22:27:57.107674",
9   "updatedAt": "2026-06-19T22:27:57.107674"
10 }
```

Actualizar producto:



Eliminar producto:



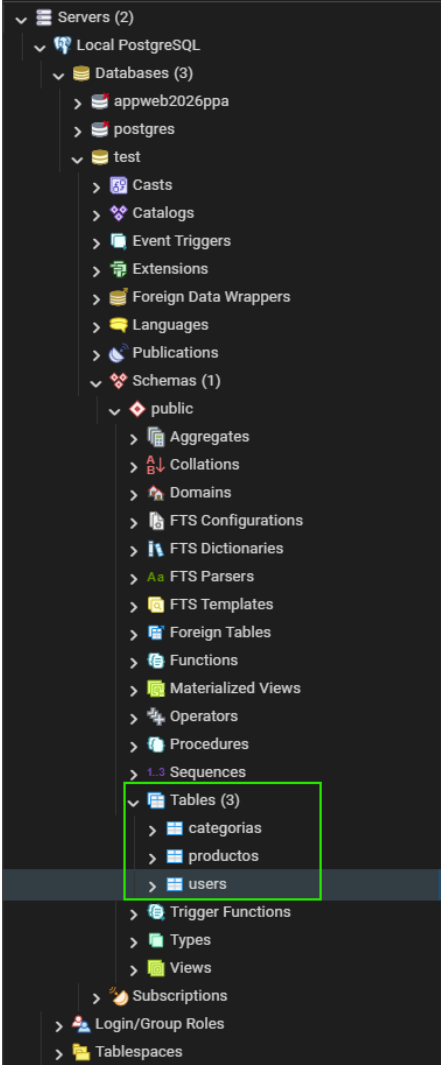
- N18 — mvn test → BUILD SUCCESS

```

PS C:\Users\Admin\Documents\VS Code\Desarrollo Web\Guia_JAVA PracticaExperimental> .\mvnw.cmd test
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.176 s -- in edu.uteq.guia
[INFO] Running edu.uteq.guia_java_practicaexperimental.service.AuthServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.309 s -- in edu.uteq.guia
[INFO] Running edu.uteq.guia_java_practicaexperimental.service.ProductoServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.164 s -- in edu.uteq.guia
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 18.069 s
[INFO] Finished at: 2026-06-19T21:41:16-05:00
[INFO] -----

```

- N19 — Tablas en BD



Showing rows: 1 to 2 | Page No: 1 of 1

	id [PK] bigint	created_at timestamp without time zone (6)	email character varying (100)	nombre character varying (100)	password character varying (255)	role character varying (50)	updated_at timestamp without time zone (6)
1	1	2026-06-19 19:47:36.12192	admin@pfc.com	Admin	\$2a\$12\$K46wK0wdYX06Zg9RnPPNF.AINoHtallVzCG5x7dBFaW11F...	ROLE_ADMIN	2026-06-19 19:47:36.12376
2	2	2026-06-19 21:54:47.567794	test@correo.com	Test User	\$2a\$12\$zopy6rC9VyznE8ETvZFMerHEqWCHbkPxorf3ZImjrArwx5jt...	ROLE_USER	2026-06-19 21:54:47.568795

7. Seguridad OWASP Aplicada

A01 — Control de Acceso Roto: Endpoints ADMIN protegidos con `@PreAuthorize("hasRole('ADMIN')")`.

A02 — Fallas Criptográficas: `BCryptPasswordEncoder(strength=12)`. JWT con HS256 y clave secreta desde variable de entorno.

A03 — Inyección: 100% queries con JPQL parametrizado. Cero concatenación SQL.

A05 — Configuración Insegura: CSRF deshabilitado. Errores sin stack traces.

A07 — Fallas de Autenticación: JWT con expiración configurable. Token inválido retorna 401.

XSS: API REST retorna JSON, no HTML. Spring MVC serializa a JSON mitigando XSS.

8. Tabla Comparativa PHP/Laravel vs Java/Spring Boot

#	PHP/Laravel	Java/Spring Boot
1. Autenticación	Sesiones + CSRF token	JWT stateless
2. Hash contraseñas	password_hash(ARGON2ID)	BCryptPasswordEncoder(12)
3. ORM	Eloquent ORM / PDO	JPA / Hibernate
4. Inyección SQL	Prepared Statements	JPQL parametrizado
5. Validación	Form Requests	Bean Validation (@Valid)
6. Rendimiento	~2,000 RPS	~8,000 RPS
7. Curva aprendizaje	Baja	Media-alta
8. Hosting Ecuador	Compartido desde \$3/mes	VPS desde \$15/mes
9. Análisis estático	PHPStan nivel 5	SonarLint + SpotBugs
10. Ecosistema	Packagist (+300K)	Maven Central (+3M)

Conclusión: Spring Boot es más adecuado para aplicaciones empresariales que requieren escalabilidad y seguridad por defecto. Laravel es superior para desarrollo rápido y prototipado con costos de hosting más bajos.

9. Preguntas de Análisis

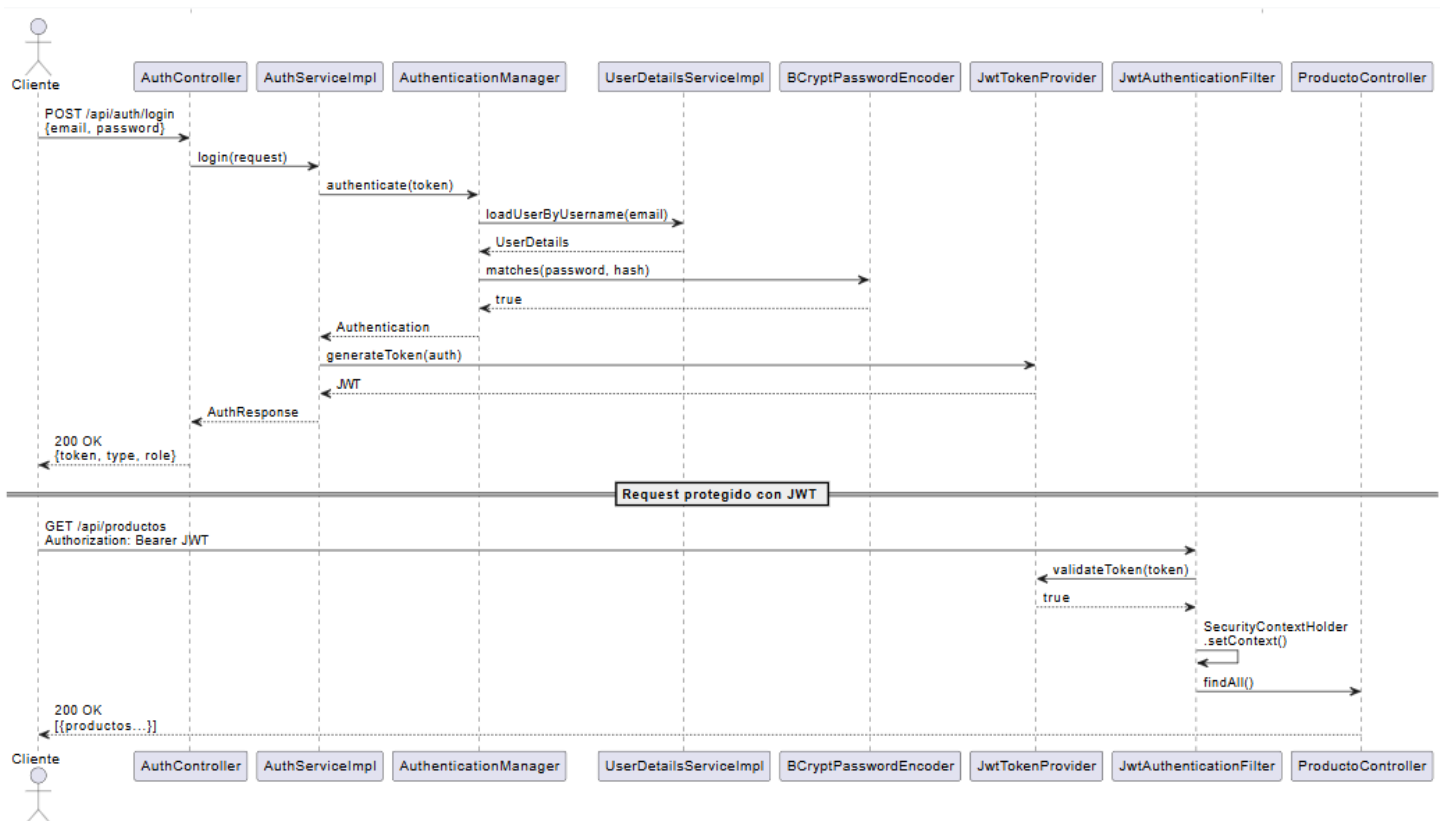
¿Spring Boot fue más seguro por defecto que PHP/Laravel?

Sí. Spring Boot con Spring Security ofrece protección CSRF por defecto, cabeceras de seguridad HTTP automáticas (HSTS, X-Frame-Options, X-Content-Type-Options), BCrypt con strength configurable sin configuración adicional, y JPQL parametrizado que previene inyección SQL sin esfuerzo del desarrollador. Laravel también es seguro, pero requiere configuración explícita del desarrollador en varios aspectos.

Recomendación tecnológica para Ecuador

Para el contexto ecuatoriano, Laravel es más accesible por hosting compartido económico (\$3-10/mes), curva de aprendizaje más baja, y comunidad latinoamericana grande. Spring Boot se recomienda para proyectos empresariales con presupuesto para VPS/cloud.

Diagrama de flujo: login → JWT → request protegido



10. Resultados de Tests

[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0

[INFO] BUILD SUCCESS

- AuthControllerTest (2 tests): Login exitoso + validación de errores
- ProductoControllerTest (2 tests): GET producto + 401 sin token
- CategoriaControllerTest (1 test): GET categorías
- AuthServiceTest (2 tests): Login + registro duplicado
- ProductoServiceTest (3 tests): findById, not found, save
- ProductoRepositoryTest (2 tests): buscarPorNombre, findByCategoriaId
- ApplicationContextTest (1 test): Contexto carga correctamente

Architecture Decision Records

Ubicados en el repositorio git hub

ADR-001 — Selección de tecnología de backend

El ADR-001 documenta la decisión de elegir Java con Spring Boot 3.x como segunda tecnología del PFC, complementando a PHP 8.x/Laravel como tecnología obligatoria. El documento evalúa tres opciones: ASP.NET Core 8, Spring Boot y Perl/CGI, descartando este último de inmediato por su modelo CGI obsoleto y la ausencia de un framework dominante en su ecosistema. La elección de Spring Boot se justifica con cinco criterios técnicos concretos: su seguridad integrada por defecto mediante BCryptPasswordEncoder y SecurityFilterChain con JWT, la diferenciación arquitectónica que ofrece al implementar autenticación stateless frente a las sesiones con estado de Laravel, el refuerzo natural de los principios SOLID a través del IoC Container, la verificación automatizada con 13 tests y BUILD SUCCESS, y el valor formativo que representa para el mercado laboral ecuatoriano de perfil empresarial.

El documento también registra las consecuencias reales de la decisión, tanto positivas como negativas. En el lado positivo, la API REST de Spring Boot puede ser consumida directamente por el frontend de la PE-U1 mediante el header Authorization con Bearer token, desacoplando completamente ambas capas. En el lado negativo, el equipo invirtió tiempo adicional en configurar Spring Security con JWT en comparación con el sistema de sesiones de Laravel, y el costo de hosting para Spring Boot en producción es significativamente mayor al de PHP en hosting compartido ecuatoriano, lo que limita su viabilidad en proyectos con presupuesto restringido.

ADR-002 — Estrategia de base de datos y acceso a datos

El ADR-002 documenta dos decisiones relacionadas: la elección de PostgreSQL 15+ como motor compartido para ambas implementaciones, y la adopción del patrón Repository con PDO en PHP y JPA/JPQL en Java como estrategia de acceso a datos. PostgreSQL fue seleccionado sobre MySQL porque el atributo `ATTR_EMULATE_PREPARES = false` funciona sin workarounds con el driver `pdo_pgsql`, el operador `ILIKE` facilita búsquedas case-insensitive en castellano, y HikariCP gestiona automáticamente el pool de conexiones en Spring Boot sin código adicional. Active Record con Eloquent fue descartado como estrategia única porque acopla el objeto de dominio al mecanismo de persistencia, viola el principio de responsabilidad única y no cumple el requisito explícito de la guía de separar interfaz y clase concreta.

El documento incluye el esquema SQL real de las tres tablas del proyecto (users, categorias, productos) e identifica una consecuencia técnica relevante: el campo `password` almacena hashes incompatibles entre plataformas, ya que PHP genera hashes Argon2id con prefijo `$argon2id$` y Java genera hashes BCrypt con prefijo `$2a$12$`, por lo que cada implementación solo puede verificar los hashes que ella misma produjo. También señala dos riesgos pendientes: la hidratación residual de modelos Eloquent dentro de `PdoProductoRepository` que introduce un acoplamiento a Laravel que debería eliminarse con DTOs puros en un refactor futuro, y la necesidad de documentar en el README la habilitación manual de la extensión `pdo_pgsql` en Laragon para garantizar la reproducibilidad del entorno.